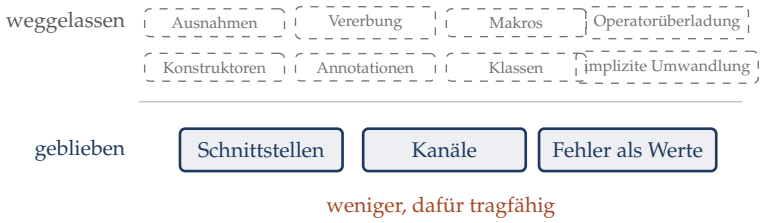
Informatik von Grund auf — Band 22

Die Kunst des Weglassens

Programmieren in Go



Was fehlt und warum
Jeder Schritt erklärt. Mit vollständig entwickelten Programmen.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 22
Die Kunst des Weglassens · Programmieren in Go

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 22: Die Kunst des Weglassens*. Fassung 1.0.0.
`https://github.com/juergenmaier2026/informatik-von-grund-auf`

Satz: LaTeX mit Palatino. Fassung 1.0.0, Oktober 2026. 118 Seiten.

Contents

Vorwort 1

I Die Auslassungen 7

1 Was fehlt und warum 9

2 Fehler als Werte 15

3 Schnittstellen 21

II Nebenläufigkeit 27

4 Goroutinen 29

5 Kanäle 35

6 Wo es schiefgeht 41

III Der Entwurf 47

7 Pakete 49

8 Zusammensetzung 55

9 Der Nullwert 61

IV Das Handwerk 67

10 Werkzeuge 69

11 Prüfen 75

12 Leistung und Speicherbereinigung 81

V Das Urteil **87**

13 Der zweite Entwurf in Go **89**

14 Wann Go und wann nicht **95**

15 Der Stand im Jahr 2026 **99**

VI Anhang **105**

A Regeln und Kenngrößen **107**

B Landkarte und Glossar **109**

C Kommentierte Quellen **111**

Vorwort: Was nicht da ist

Die meisten Sprachen erklärt man über das, was sie können. *Go erklärt man über das, was es weglässt. Jede Auslassung war eine Entscheidung mit einer Begründung — und über sie zu streiten ist lehrreicher als über jede vorhandene Eigenschaft.*

Acht Auslassungen

Fehlt	Begründung	Ersatz
Ausnahmen	**Fehlerpfad sichtbar**	**Rückgabewert**
Vererbung	Hierarchien altern schlecht	**Zusammen-setzung**
Klassen	Methoden reichen	Typ mit Methoden
Konstruktoren	**Nullwert brauchbar**	Kap. 9
Makros	Band 21, Kap. 9	keiner
Operator-überladung	**Lesbarkeit**	keiner
Implizite Umwandlung	Band 21, Kap. 8	ausdrücklich
Generizität bis 2022	**Zeit gebraucht**	seither da

Drei dieser Auslassungen sind unmittelbare Antworten auf Erfahrungen aus Band 21 — Makros, implizite Umwandlung und die stillen Fehlerpfade. Die letzte Zeile zeigt, dass eine Auslassung auch zurückgenommen werden kann, wenn man zwölf Jahre über die richtige Form nachgedacht hat.

Der gemeinsame Nenner.

Die Leitfrage	Was kostet dieses Sprachmittel den *Leser*?
Die Antwort	**Wenn mehr, als es dem Schreiber nützt: weglassen**
Die Folge	Eine kleine Sprache, die man in zwei Tagen kennt

Das ist eine ungewöhnliche Gewichtung, und sie folgt aus einer Beobachtung, die Band 20 beziffert hat: Man liest anderthalbmal so viel, wie man schreibt. Wer für den Leser entwirft, entwirft für die häufigere Tätigkeit.

Was das Weglassen kostet

Auslassung	Preis	
Keine Ausnahmen	viel `if err != nil`	**Wiederholung**
Keine Vererbung	Umstellung nötig	Gewöhnung
Keine Überladung	`Add`, `AddInt`, `AddAll`	Namenswucher
Lange ohne Generizität	**Code verdoppelt**	behoben

Die erste Zeile ist die meistgenannte Kritik an Go, und sie ist berechtigt: Der Fehlerpfad ist sichtbar, weil er ausgeschrieben wird — und das sind viele Zeilen. Kapitel 2 rechnet aus, wie viele, und stellt die Frage, ob der Handel sich lohnt.

Dieser Band verteidigt Go nicht. Er nimmt jede Auslassung ernst, nennt den Preis und lässt den Leser urteilen — so wie Band 21 die Kosten von C aufgeschrieben hat, ohne sie zu beschönigen.

Der dritte Pol

	Python	C	Go
Zeigt	**was man ausdrücken kann**	**was geschieht**	**was man weglässt**
Speicher	verwaltet	selbst	verwaltet
Zeitschranken	nein	**ja**	nein
Nebenläufigkeit	angeflanscht	Handarbeit	**Entwurfsgegenstand**
Sprachumfang	groß	klein	**sehr klein**

Die vierte Zeile ist der sachliche Grund für diesen Band: Nebenläufigkeit ist in Python angeflanscht und in C Handarbeit mit Fußangeln. In Go ist sie das, worum die Sprache herum entworfen wurde — und deshalb lässt sie sich hier ohne schlechte Beispiele erklären.

Das Vorhaben, fortgesetzt.

Der Bestandteil	Die Übertragungsschicht zwischen Gerät und Klinik
Warum passt Go	**Viele gleichzeitige Verbindungen, weiche Schranken**
Was hinzukommt	Netz, Ausfälle, Gegendruck, Abbruch

Das ist der Bestandteil, den Band 21, Kapitel 14 an Go verwiesen hat. Tausend Geräte, die gleichzeitig Messwerte senden, ein Netz, das ausfällt, und ein Empfänger, der langsamer ist als die Sender — die Aufgabe, für die Go gebaut wurde.

Was dieses Buch will

Erstens: die Auslassungen begründen und ihren Preis nennen.

> *Zu jeder fehlenden Eigenschaft gehört die Frage, was man dafür bekommt und was man dafür bezahlt. Wer nur den Gewinn nennt, betreibt Werbung; wer nur den Preis nennt, hat nicht verstanden, worum es ging.*

Zweitens: Nebenläufigkeit richtig lehren.

Goroutinen sind billig, und genau deshalb ist es leicht, Millionen davon falsch zu benutzen. Kapitel 6 behandelt die drei Fehlerklassen, die dabei entstehen — und alle drei sind in Go leichter zu machen als in C.

Drittens: sagen, wann Go nicht passt.

Die Speicherbereinigung schließt harte Zeitschranken aus. Das ist keine Kleinigkeit, sondern eine Grenze — und Kapitel 14 nennt sie zusammen mit den anderen.

Drei Warnungen

Erstens: Einfach ist nicht dasselbe wie leicht.

> *Go hat wenige Sprachmittel, und daraus folgt nicht, dass Programme einfach werden. Die Komplexität der Aufgabe verschwindet nicht — sie wandert aus der Sprache in den Code. Ob das besser ist, entscheidet sich am Einzelfall.*

Zweitens: Goroutinen sind nicht kostenlos.
Sie sind billig — einige Kilobyte statt einiger Megabyte — und billig ist nicht null. Eine Goroutine, die nie endet, ist ein Leck, und Lecks dieser Art sind schwerer zu finden als Speicherlecks in C.
Drittens: Der Formatierer beendet Stildiskussionen, nicht Entwurfsdiskussionen.

`gofmt` legt fest, wie Code aussieht, und sagt nichts darüber, ob er gut ist. Alles aus Band 20 — Zerlegung, Benennung, Datenmodell — bleibt zu tun.

Zum Aufbau

- **Teil I** (Kap. 1–3): Die Auslassungen. Warum sie fehlen, **Fehler als Werte**, Schnittstellen.
- **Teil II** (Kap. 4–6): Nebenläufigkeit. Goroutinen, Kanäle, **wo es schiefgeht**.
- **Teil III** (Kap. 7–9): Der Entwurf. Pakete, Zusammensetzung, **der Nullwert**.
- **Teil IV** (Kap. 10–12): Das Handwerk. Werkzeuge, Prüfen, **Leistung**.
- **Teil V** (Kap. 13–15): Das Urteil. Der zweite Entwurf, **wann Go**, der Stand 2026.

Eine Sprache gegen einen Bauvorgang

Go entstand nicht aus einer Idee über Sprachen, sondern aus einem Ärgernis.

Der Anlass	Ein Bauvorgang, der bei einem großen Anbieter **fünfundvierzig Minuten** dauerte
Die Ursache	Kopfdateien, die einander einschließen — Band 21, Kap. 11
Die Entscheidung	**Eine Sprache, deren Abhängigkeiten ein Graph ohne Zyklen sind**

Aus dieser einen Anforderung folgen mehrere Sprachentscheidungen: keine Kopfdateien, keine zirkulären Einschlüsse, ein Übersetzer, der jede Datei genau einmal liest. Der Bauvorgang dauert seither Sekunden statt Minuten.

Was daraus noch folgte.

Entscheidung	Folge	
Keine Kopfdateien	**schneller Bau**	
Unbenutzte Einschlüsse sind Fehler	sauber	**lästig beim Probieren**
Ein Formatierer	keine Stildebatten	
Alles in einer Binärdatei	**einfache Auslieferung**	

Die zweite Zeile ist die umstrittenste: Ein nicht benutzter Einschluss ist kein Hinweis, sondern ein Übersetzungsfehler. Beim Probieren ist das lästig; im Bestand hält es die Abhängigkeiten sauber. Man kann beides für richtig halten, und darüber lohnt der Streit.

> *Bemerkenswert ist die Herkunft: Nicht eine Theorie über Sprachen, sondern die Erfahrung mit einem großen Bestand hat die Entscheidungen bestimmt. Das erklärt, warum Go dort überzeugt, wo es um viele Zeilen und viele Beteiligte geht — und weniger dort, wo es um Ausdrucksstärke geht.*

Eine Eigenschaft wegzulassen ist eine Entscheidung,
die man begründen muss.
Eine hinzuzufügen auch —
nur verlangt das niemand.

— der Leitgedanke dieses Bandes.
Was fehlt, sieht man sofort.
Was zu viel ist, merkt man erst nach Jahren.

Teil I

Die Auslassungen

Kapitel 1

Was fehlt und warum

Der Maßstab: Kosten für den Leser

Sprachmittel	Nutzt dem Schreiber	Kostet den Leser
Operator-überladung	kürzer	**was tut +?**
Vererbung	Wiederver-wendung	**welche Methode gilt?**
Makros	Verallgemei-nerung	was steht wirklich da?
Ausnahmen	knapper Ablauf	**wo geht es weiter?**
Implizite Umwandlung	bequem	welcher Typ?

Jede Zeile beschreibt einen Handel: Der Schreiber spart Zeichen, der Leser verliert Gewissheit. Go entscheidet diesen Handel durchgehend zugunsten des Lesers — und das ist die einzige Regel, aus der sich alle acht Auslassungen ableiten lassen.

Die Begründung dahinter.

Die Lage	Ein Bestand mit Millionen Zeilen und Tausenden Beteiligten
Die Beobachtung	**Code wird öfter gelesen als geschrieben**
Die Folgerung	Sprachmittel, die das Lesen erschweren, kosten mehr, als sie bringen

Die zweite Zeile ist die Zahl aus Band 20: dreißig Prozent Lesen, zwanzig Prozent Schreiben. Die Folgerung ist ungewöhnlich, weil fast alle anderen Sprachen den umgekehrten Handel eingehen.

Wo derselbe Maßstab anders entscheidet

Lage	Maßstab passt?	
Großer Bestand, viele Leute	**ja**	
Bibliothek für viele Nutzer	ja	
Ein Skript, einmal	**nein**	Ausdruck zählt
Mathematische Bibliothek	nein	**Überladung wäre richtig**

Die letzte Zeile ist das ehrlichste Gegenbeispiel: Für Vektorrechnung ist `a.Add(b.Mul(c))` schlechter lesbar als `a + b*c` — hier kostet die Auslassung genau das, was sie schützen sollte. Go ist für diese Aufgabe die falsche Sprache, und das ist kein Mangel, sondern eine Festlegung.

Der Formatierer

	Mit Optionen	Ohne
Stildiskussionen	**bleiben**	**beendet**
Anpassbar	ja	nein
Fremder Code lesbar	je nach Projekt	**immer gleich**
Zeilenunterschiede	Formatierung	nur Inhalt

Die Entscheidung, dem Formatierer keine Optionen zu geben, wirkt kleinlich und ist die folgenreichste des ganzen Werkzeugkastens. Sie beendet eine Diskussion, die in anderen Sprachen bei jedem neuen Projekt neu geführt wird — und die niemand gewinnen kann, weil es keinen sachlichen Maßstab gibt.

Die vierte Zeile ist der praktische Gewinn beim Durchsehen: Ein Unterschied zeigt nur, was sich geändert hat — nicht, dass jemand anders einrückt. Band 17, Kapitel 13 fordert genau das und kann es nicht durchsetzen.

Was der Formatierer nicht leistet

Frage	Beantwortet?	
Wie sieht es aus?	**ja**	
Sind die Namen gut?	nein	Band 20, Kap. 5
Ist die Zerlegung richtig?	**nein**	
Stimmt das Datenmodell?	**nein**	**Band 20, Kap. 2**

Ein einheitliches Aussehen erzeugt den Eindruck von Ordnung, und der kann täuschen. Alles, was Band 20 behandelt, bleibt zu tun — der Formatierer nimmt nur die Diskussion ab, die ohnehin keine Substanz hatte.

Übungsaufgaben

(a) Bewerten Sie eine Auslassung an einem Beispiel.

(b) Rechnen Sie die Kosten fehlender Generizität aus.

(c) Untersuchen Sie, was die Sprachgröße bewirkt.

Lösung

(a) Eine Auslassung bewerten.
Der Fall. Vererbung, am Befundmodell aus Band 20.

```
Befund
Laborbefund
Klinischchemie
Mikrobiologie
Bildbefund
Funktionsbefund
```

Neue Anforderung	Passt in die Hierarchie?	
Bakteriologie mit Zahlwert	knapp	
Blutgasanalyse, bettseitig	**zwei Zweige**	**Bruch**
Befund aus Fremdhaus	quer	
Bild und Zahlen zugleich	**beides**	Mehrfach-vererbung

Die Hierarchie ist nach zwei Jahren richtig und nach fünf falsch — weil die Wirklichkeit sich nicht in einen Baum fügt. Das ist die Begründung für die Auslassung, und sie stammt aus Erfahrung, nicht aus Theorie.

Ohne Vererbung.

```
type Befund interface {
Patient() PatientID
Abnahme() time.Time
}

type Messbar interface {
Wert() Ergebnis
Bereich() Referenz
}

type Bebildert interface { Bilder() []BildID }
```

Drei kleine Schnittstellen statt eines Baums. Ein Befund mit Bild und Zahlen erfüllt beide — ohne Mehrfachvererbung, ohne Umbau der Hierarchie. Kapitel 3 führt das aus.

	Hierarchie	Schnitt-stellen
Neue Art einfügen	Umbau	**nichts**
Zwei Eigenschaften	unmöglich	ja
Wo ist die Methode?	**suchen**	**beim Typ**
Gemeinsamer Code	geerbt	wiederholt

Die letzte Zeile ist der Preis, und er ist echt: Gemeinsamer Code wird nicht geerbt, sondern eingebettet oder wiederholt. Kapitel 8 zeigt, wie das aussieht — und wo es lästig bleibt.

(b) Kosten fehlender Generizität.
Die Lage vor 2022. Eine Filterfunktion für drei Typen.

Weg	Zeilen	Typprüfung	Dauer
Dreimal schreiben	**42**	**voll**	1,0
Über `interface{}`	16	**keine**	3,4
Codeerzeugung	18	voll	1,0
Typparameter	**16**	**voll**	**1,0**

Zwölf Jahre lang musste man zwischen den ersten drei Zeilen wählen — und alle drei sind unbefriedigend. Die vierte Zeile zeigt, dass die Auslassung tatsächlich etwas gekostet hat und dass ihre Rücknahme richtig war.

Bemerkenswert ist die Begründung für das lange Warten: Man wollte keine Lösung, die die Übersetzungszeit oder die Lesbarkeit beschädigt. Ob das zwölf Jahre wert war, kann man bestreiten — dass es eine Entscheidung und kein Versäumnis war, nicht.

(c) Was die Sprachgröße bewirkt.

Sprache	Schlüssel-wörter	Spezifikation	Einarbeitung
Go	**25**	≈ 90 S.	**2–5 Tage**
C	32–44	≈ 550 S.	2–4 Wochen
Python	35	≈ 200 S.	1–2 Wochen
C++	≈ 95	≈ 1800 S.	Monate

Die letzte Spalte ist die interessante: Ein erfahrener Programmierer liest die Go-Spezifikation an einem Nachmittag und kann danach jeden Go-Code lesen. Das ist kein Anfängervorteil, sondern einer für den, der fremden Code beurteilen muss — Band 20, Kapitel 8.

Die Kehrseite.

Folge	Bewertung	
Wenig zu lernen	gut	
Wenig auszudrücken	**einschränkend**	
Wiederholung im Code	**häufig**	**Kap. 2**
Weniger Überraschungen	gut	

Die dritte Zeile ist die ehrlichste Kritik: Go-Code enthält viel Wiederholung, und die Sprache bietet wenig dagegen. Wer das als Nachteil empfindet, hat recht — und wer die Wiederholung der Undurchschaubarkeit vorzieht, auch.

Auslassungen sind nicht neutral

Auslassung	Wirkt gegen	
Keine Ausnahmen	knappen Code	**Kap. 2**
Keine Überladung	Mathematik	
Keine Vererbung	Rahmenwerke	**schwer zu bauen**
Kein Makrosystem	Sprach-erweiterung	

Die dritte Zeile ist die folgenreichste: Ein Rahmenwerk, das den Ablauf übernimmt und den eigenen Code aufruft, lässt sich in Go schwer bauen. Deshalb gibt es in Go Bibliotheken statt Rahmenwerken — und das ist eine Wirkung der Sprachentscheidung auf die Baukultur.

Wer Go beurteilt, sollte diese Wirkungen kennen. Eine Sprache formt, was in ihr entsteht — Conways Gesetz aus Band 17, angewandt auf Sprachentwurf statt auf Organisationen.

Pike[19] beschreibt, aus welchen Erfahrungen mit großen Beständen die Entscheidungen entstanden; die Entwurfsdokumente[12] begründen jede Auslassung einzeln; die Sprichwortsammlung[20] verdichtet die Haltung auf zwanzig Sätze.

Kapitel 2

Fehler als Werte

Der Handel

	Ausnahmen	Werte
Fehlerpfad	**unsichtbar**	**ausgeschrieben**
Ignorierbar	nein	ja — mit _
Zeilen	wenige	**viele**
Wo geht es weiter?	**irgendwo oben**	nächste Zeile
Aufräumen	`finally`	`defer`

Die erste und die dritte Zeile sind die beiden Seiten desselben Handels: Der Fehlerpfad wird sichtbar, weil er dasteht — und deshalb sind es viele Zeilen. Man kann nicht beides haben.

Die Form.

```
wert, err := lies(pfad)
if err != nil {
return nil, fmt.Errorf("lesen von %s:  %w", pfad, err)
}
```

Vier Zeilen für eine Operation. Die dritte ist die wichtige: Sie fügt Zusammenhang hinzu und behält den ursprünglichen Fehler — damit entsteht eine Kette, die den Weg durch das Programm beschreibt.

Was die Fehlerkette leistet

Meldung	`uebertragung: geraet 4711: lesen von /dev/ttyS0: keine Berechtigung`
Was sie sagt	**Wo, für wen, was, warum — in einer Zeile**
Prüfbar	`errors.Is` und `errors.As` finden die Ursache in der Kette

Das ist der Ersatz für die Ablaufverfolgung einer Ausnahme — und in einer Hinsicht besser: Die Kette enthält, was der Programmierer für wichtig hielt, nicht jeden Zwischenaufruf. Sie ist kürzer und aussagekräftiger, wenn sie gepflegt wird.

Der Haken steht in der Bedingung: wenn sie gepflegt wird. Ein blankes `return err` ohne Zusammenhang erzeugt am Ende die Meldung "keine Berechtigung" ohne jeden Hinweis, wo.

`defer` und was es löst

	C, Band 21	Go
Aufräumen	**ein Ausgang mit `goto`**	**`defer` bei der Belegung**
Abstand	Funktionsende	**eine Zeile**
Bei frühem Verlassen	Sprung nötig	automatisch
Vergessbar	ja	kaum

Die zweite Zeile ist der eigentliche Gewinn: Die Freigabe steht neben der Belegung, nicht vierzig Zeilen später. Damit sieht man beim Lesen sofort, ob eine Belegung eine Freigabe hat — das quadratische Problem aus Band 21, Kapitel 5 verschwindet.

```
f, err := os.Open(pfad)
if err != nil { return err }
defer f.Close()            // steht hier, laeuft am Ende
```

Drei Fallen bei `defer`

Falle	Folge	
`defer` in der Schleife	**läuft erst am Funktionsende**	**Dateien häufen sich**
Argumente sofort ausgewertet	alter Wert	überraschend
Fehler von `Close` ignoriert	**Daten verloren**	**Band 21, Kap. 5**

Die erste Falle ist die häufigste: Eine Schleife über zehntausend Dateien mit `defer f.Close()` hält alle zehntausend offen, bis die Funktion endet. Die Abhilfe ist eine eigene Funktion je Durchlauf — oder das Schließen von Hand.

Die dritte Falle ist dieselbe wie in C: `Close` kann fehlschlagen, weil erst dort geschrieben wird. Wer beim Schreiben `defer f.Close()` benutzt, erfährt nie, dass die Datei unvollständig ist.

Übungsaufgaben

(a) Zählen Sie die Fehlerzeilen.

(b) Entwerfen Sie eigene Fehlertypen.

(c) Bestimmen Sie, wann `panic` richtig ist.

Lösung

(a) Fehlerzeilen zählen.
Der Fall. Die Übertragungsschicht, dieselbe Aufgabe in drei Sprachen.

	Zeilen	davon Fehler	Anteil
Python	310	42	14 %
Go	**480**	**146**	**30 %**
C	890	394	44 %

Dreißig Prozent des Go-Codes sind Fehlerbehandlung — mehr als doppelt so viel wie in Python. Das ist die Kritik aus dem Vorwort in Zahlen, und sie trifft zu.

Die Gegenrechnung.

	Python	Go	
Unbehandelte Fehler	**11**	**2**	
Davon im Betrieb	4	0	
Fehlermeldung nützlich	teils	**fast immer**	**Kette**
Zeit bis Ursache	≈ 45 min	**≈ 6 min**	

Elf unbehandelte Fehler gegen zwei — weil in Python eine Ausnahme ungefangen durchrutschen kann, ohne dass es beim Schreiben auffällt. In Go steht der Fehlerwert da und wird vom Prüfer bemängelt, wenn man ihn ignoriert.

Die letzte Zeile ist der Gewinn, für den man die hundertvier zusätzlichen Zeilen bezahlt: sechs Minuten statt fünfundvierzig bis zur Ursache. Ob sich das lohnt, hängt davon ab, wie oft man sucht — und Band 20 hat gezeigt, dass es ein Viertel der Zeit ist.

(b) Eigene Fehlertypen.

```
var ErrGeraetUnbekannt = errors.New("geraet unbekannt")

type WertFehler struct {
Geraet  GeraetID
Zeile   int
Ursache error
}

func (e *WertFehler) Error() string {
return fmt.Sprintf("geraet %s zeile %d:  %v",
e.Geraet, e.Zeile, e.Ursache)
}
func (e *WertFehler) Unwrap() error { return e.Ursache }
```

Art	Wofür	
Wächter (`Err...`)	**Vergleich**	`errors.Is`
Eigener Typ	**Zusatzdaten**	`errors.As`
Nur Text	einfache Fälle	nicht prüfbar

Die Wahl folgt aus der Frage, was der Aufrufer tun soll. Wenn er nur melden will, genügt Text. Wenn er unterscheiden muss, braucht er einen Wächter. Wenn er Einzelheiten braucht — welche Zeile, welches Gerät — einen eigenen Typ.

Der Fehler, den man dabei macht: einen eigenen Typ zu bauen, bevor irgendein Aufrufer die Einzelheiten braucht. Die Regel der drei Fälle aus Band 20, Kapitel 14 gilt auch hier.

(c) Wann `panic` richtig ist.

Lage	`panic`?	
Datei fehlt	**nein**	**erwarteter Fall**
Netz weg	nein	
Ungültige Eingabe	nein	
Unmöglicher Zustand	**ja**	**Programmfehler**
Beim Programmstart	ja	Konfiguration fehlt
Über Paketgrenze	**nie**	**fangen und wandeln**

Dieselbe Unterscheidung wie in Band 17, Kapitel 6: `panic` ist die Zusicherung, kein Fehlerweg. Sie bedeutet "hier ist ein Programmfehler", nicht "etwas ist schiefgegangen".

Die letzte Zeile ist die Regel für Bibliotheken: Ein `panic`, das die Paketgrenze überschreitet, zwingt jeden Aufrufer zu einem Fangblock — also zu genau dem, was die Sprache vermeiden wollte. An der Grenze wird gefangen und in einen Fehlerwert verwandelt.

Die Wiederholung ist der Preis

Vorschlag	Hilft?	
Hilfsfunktion `check`	**nein**	versteckt den Pfad
Fehler sammeln	teils	bei Formularen
Ablauf umbauen	**ja**	**weniger Aufrufe**
Damit leben	oft	

Die dritte Zeile ist die einzige echte Abhilfe und keine Sprachfrage: Eine Funktion mit zwölf Fehlerprüfungen tut zu viel. Wer sie zerlegt, hat weniger Prüfungen je Funktion — Band 20, Kapitel 5 in Go-Anwendung.

Die letzte Zeile ist die ehrlichste: Ein Teil der Wiederholung bleibt, und man gewöhnt sich daran oder nicht. Wer sie unerträglich findet, sollte Go nicht wählen — und das ist ein legitimes Urteil.

Die Beiträge zur Fehlerbehandlung[25] erklären Kette, `Is` und `As`; Donovan und Kernighan[6] führen die Formen mit ihren Abwägungen vor; Cheney[2] hat die Diskussion über Fehlerbehandlung in Go am gründlichsten geführt.

Kapitel 3

Schnittstellen

Erfüllung ohne Erklärung

	Ausdrücklich	Nach Gestalt
Typ nennt sie	ja	**nein**
Nachträglich möglich	nein	**ja**
Fremde Typen	nein	ja
Wer legt fest?	**der Anbieter**	**der Benutzer**

Die letzte Zeile ist die eigentliche Umkehrung. In Java oder C++ entscheidet der, der einen Typ schreibt, welche Schnittstellen er erfüllt. In Go entscheidet der, der ihn benutzt, welche Schnittstelle er braucht — und definiert sie dort, wo sie gebraucht wird.

Die Folge für den Entwurf.

Die Regel	Schnittstellen gehören zum Benutzer, nicht zum Anbieter
Die Größe	**Je kleiner, desto brauchbarer**
Der Maßstab	Eine Methode ist oft genug

Das ist die Umkehrung dessen, was man aus anderen Sprachen kennt: Dort baut man große Schnittstellen im Voraus. Hier schreibt der Aufrufer eine Schnittstelle mit genau den zwei Methoden, die er braucht — und jeder Typ, der sie hat, passt.

Die kleinste nützliche Schnittstelle

```
type Writer interface {
Write(p []byte) (n int, err error)
}
```

Erfüllt von	Erwartet?	
Datei	ja	
Netzverbindung	ja	
Puffer im Speicher	ja	**für Prüfungen**
Prüfsummenrechner	ja	
Komprimierer	**ja**	**lässt sich stapeln**

Eine Methode, und jeder Empfänger von Bytes passt hinein. Die dritte Zeile ist der Prüfgewinn: Man ersetzt die Netzverbindung durch einen Speicherpuffer und prüft ohne Netz — ohne Attrappe, ohne Rahmenwerk.

Die Schnittstelle beim Aufrufer

Der übliche Fehler.

```
// paket geraet - so nicht
type GeraetLeser interface {
Lies() (Messwert, error)
Kalibriere() error
Status() Status
Zuruecksetzen() error
}
```

Vier Methoden, weil das Gerät sie hat. Wer nur lesen will, muss trotzdem alle vier bereitstellen — und eine Prüfattrappe braucht vier Methoden, von denen drei leer sind.

Beim Aufrufer, so klein wie nötig.

```
// paket uebertragung
type Quelle interface { Lies() (Messwert, error) }

func Uebertrage(q Quelle, z Ziel) error { ... }
```

	Beim Anbieter	Beim Aufrufer
Methoden	4	**1**
Attrappe	22 Zeilen	**4** Zeilen
Abhängigkeit	auf `geraet`	**keine**
Andere Quellen	Umbau	passen

Die dritte Zeile ist der Gewinn, den man erst nach Monaten bemerkt: Das Übertragungspaket kennt das Gerätepaket nicht. Damit lässt es sich einzeln prüfen, einzeln übersetzen und einzeln wiederverwenden — Parnas' Kriterium aus Band 17 in seiner schärfsten Form.

Wo die Gestaltprüfung überrascht

Fall	Wirkung	
Gleicher Name, andere Bedeutung	**erfüllt versehentlich**	**`Close`, `Read`**
Methode umbenannt	Erfüllung fällt weg	Fehler erst beim Übersetzen
Kein Bezug im Code	**schwer zu finden**	**wer erfüllt sie?**

Die dritte Zeile ist der Preis für die Umkehrung: Es steht nirgends, welche Typen eine Schnittstelle erfüllen. Man findet es nur mit Werkzeugen — und beim Lesen fehlt der Hinweis, den ein ausdrückliches `implements` gäbe.

Die übliche Abhilfe ist eine Zusicherung beim Übersetzen: `var _ Quelle = (*Geraet)(nil)`. Sie kostet eine Zeile, erzeugt keinen Code und dokumentiert die Absicht dort, wo sie hingehört.

Übungsaufgaben

(a) Schneiden Sie Schnittstellen zu.

(b) Untersuchen Sie die Nullwertfalle bei Schnittstellen.

(c) Bestimmen Sie die Kosten der Indirektion.

Lösung

(a) Schnittstellen zuschneiden.
Die Übertragungsschicht, Bedarf je Bestandteil.

Bestandteil	Braucht	Schnittstelle
Übertrager	**lesen**	1 Methode
Überwachung	Status	1 Methode
Wartungs-werkzeug	**alles**	4 Methoden
Prüfung	lesen	1 Methode

Vier Bestandteile, vier verschiedene Bedürfnisse — und nur eines braucht alle vier Methoden. Wer eine gemeinsame Schnittstelle für alle baut, zwingt drei Bestandteile zu einer Abhängigkeit, die sie nicht brauchen.

Die Wirkung auf die Abhängigkeiten.

	Eine große	Vier kleine
Pakete mit Abhängigkeit	4	**1**
Übersetzung bei Änderung	4 Pakete	**1**
Prüfattrappen gesamt	88 Zeilen	**16**

Die zweite Zeile wirkt auf die Bauzeit und damit auf die Rückmeldedauer aus Band 17, Kapitel 10. Kleine Schnittstellen sind nicht nur eine Entwurfsfrage — sie halten den Bauvorgang schnell.

(b) Die Nullwertfalle.

```
var q *Geraet = nil
var s Quelle = q
if s != nil { ...  }    // wahr!  - die Falle
```

Bestandteil	Wert	
Typ	`*Geraet`	**nicht null**
Wert	`nil`	
Schnittstelle	**nicht `nil`**	weil der Typ da ist

Eine Schnittstelle besteht aus zwei Teilen: Typ und Wert. Sie ist nur dann `nil`, wenn beide leer sind. Ein Nullzeiger eines konkreten Typs erzeugt eine Schnittstelle, die nicht `nil` ist und beim Methodenaufruf abstürzt.

Wo das auftritt.

```
func hole() error {
var e *MeinFehler = nil
if schiefgelaufen { e = &MeinFehler{} }
return e            // FALSCH: nie nil
}
```

Der häufigste Fall in der Praxis — und er macht jede Fehlerprüfung beim Aufrufer wirkungslos, weil `err != nil` immer wahr ist. Die Abhilfe ist einfach: `error` als Typ der Variablen, nicht den konkreten Typ.

Erkennbar durch	**Findet es?**	
Lesen	selten	
Übersetzer	**nein**	
`go vet`	**ja**	**seit Jahren**
Prüfung	ja	wenn vorhanden

(c) Kosten der Indirektion.

Aufruf	**Dauer**	**Einbettbar**	
Direkt, konkret	**1,0** ns	**ja**	
Über Schnittstelle	2,4 ns	teils	
In enger Schleife	**+140** %		spürbar
Bei Netzarbeit	< 0,01 %		belanglos

Die letzten beiden Zeilen sind dieselbe Zahl in zwei Zusammenhängen. In einer Schleife über Millionen Werte kostet die Indirektion die Hälfte der Laufzeit; bei einer Übertragung, die auf das Netz wartet, ist sie nicht messbar.

Daraus folgt die Regel: Schnittstellen an den Rändern, konkrete Typen im Inneren enger Schleifen. Das ist derselbe Gedanke wie in Band 21, Kapitel 14 — die Grenze verläuft entlang grober Operationen.

Schnittstellen im Voraus sind fast immer falsch

Anlass	Berechtigt?	
Zwei Umsetzungen vorhanden	**ja**	
Prüfung braucht Ersatz	ja	
“Könnte später nützlich sein”	**nein**	**Band 20, Kap. 14**
“Guter Stil”	nein	

Eine Schnittstelle mit genau einer Umsetzung ist eine zusätzliche Ebene ohne Gegenwert — man muss beim Lesen einen Sprung mehr machen, um zu sehen, was geschieht. Weil Go Schnittstellen nachträglich zulässt, kostet das Warten nichts.

Das ist der Vorteil der Gestaltprüfung, den man am leichtesten verschenkt: Man kann sie einführen, wenn man sie braucht — und muss es deshalb nicht vorher tun.

Parnas[17] begründete, warum die Schnittstelle verbergen und nicht zeigen soll; Ousterhout[16] fordert tiefe Schnittstellen — wenig nach außen, viel dahinter; Donovan und Kernighan[6] führen die Gestaltprüfung mit ihren Feinheiten vor.

Teil II

Nebenläufigkeit

Kapitel 4

Goroutinen

Was sie kosten

	Systemfaden	Goroutine	Faktor
Keller anfangs	≈ 8 MB	**2 KB**	4000
Erzeugen	≈ 12 µs	**0,3** µs	40
Umschalten	≈ 1,5 µs	**0,1** µs	15
Praktisch möglich	≈ 10^4	**10^6**	

Die erste Zeile ist der Kern: Ein Systemfaden reserviert acht Megabyte Kellerbereich, eine Goroutine zwei Kilobyte, die bei Bedarf wachsen. Deshalb sind eine Million Goroutinen möglich und eine Million Fäden nicht.

Was das für die Übertragungsschicht bedeutet.

Bauform	Bei 1000 Geräten	
Ein Faden je Gerät	≈ 8 GB Keller	unmöglich
Ereignisschleife	ein Faden	**Ablauf zerrissen**
Eine Goroutine je Gerät	**≈ 20 MB**	**Ablauf zusammenhängend**

Die zweite Zeile ist die Bauform, die man in anderen Sprachen wählen muss: eine Ereignisschleife mit Rückrufen, in der ein logischer Ablauf über zehn Funktionen verteilt liegt. Die dritte Zeile schreibt denselben Ablauf hintereinander — lesbar wie ein sequentielles Programm.

Nebenläufigkeit ist nicht Parallelität

	Nebenläufigkeit	Parallelität
Ist	**eine Entwurfsform**	**eine Ausführungsform**
Betrifft	**Zerlegung**	Hardware
Nützt bei	Warten	**Rechnen**
Auf einem Kern	sinnvoll	unmöglich

Pikes Unterscheidung ist die nützlichste des Kapitels: Nebenläufigkeit heißt, ein Programm in unabhängig fortschreitende Teile zu zerlegen. Ob diese Teile gleichzeitig laufen, ist eine andere Frage — und für die Übertragungsschicht, die überwiegend wartet, die unwichtigere.

Wer beendet die Goroutine?

Die Regel	Wer eine Goroutine startet, muss wissen, wann sie endet
Der Grund	**Es gibt keinen Weg, sie von außen abzubrechen**
Das Mittel	Ein Abbruchsignal, das sie selbst beachtet

Die zweite Zeile überrascht viele: Go bietet kein `kill` für Goroutinen — absichtlich, weil ein Abbruch von außen den Zustand in unbekanntem Zustand hinterlässt. Eine Goroutine endet, wenn ihre Funktion endet, und sonst nie.

```
func lauscher(ctx context.Context, g Geraet, aus chan<-
Messwert) {
for {
m, err := g.Lies()
if err != nil { return }
select {
case aus <- m:
case <-ctx.Done():    // Abbruch beachtet
return
}
}
}
```

Der Abbruchkontext ist das erste Argument — eine Vereinbarung, die durchgehalten wird. Ohne den `select` würde die Goroutine ewig auf einen Abnehmer warten, auch wenn niemand mehr zuhört. Das ist das Leck aus Kapitel 6.

Die drei Enden

Ende	Sauber?	
Funktion endet	**ja**	**der Normalfall**
Abbruch über Kontext	ja	
`panic`	**reißt alles mit**	**das ganze Programm**
Programm endet	abrupt	kein Aufräumen

Die dritte Zeile ist eine harte Eigenschaft: Ein `panic` in irgendeiner Goroutine beendet das gesamte Programm — es gibt keine Kapselung je Goroutine. Wer tausend Geräteverbindungen bedient, verliert alle, wenn eine davon einen Programmfehler auslöst.

Die Abhilfe ist ein Fangblock am Anfang jeder langlebigen Goroutine: `defer func() { if r := recover(); r != nil { melde(r) } }()`. Das ist eine der wenigen Stellen, an denen `recover` richtig ist.

Übungsaufgaben

(a) Rechnen Sie die Bauformen gegeneinander.

(b) Begrenzen Sie die Zahl der Goroutinen.

(c) Untersuchen Sie das Fadenmodell darunter.

Lösung

(a) Bauformen gegeneinander.
Die Aufgabe. 1000 Geräte, je 500 Messwerte je Sekunde, Weiterleitung über das Netz.

Bauform	Zeilen	Speicher	Durchsatz
Ereignisschleife	**680**	14 MB	**510** k/s
Fadenbündel	420	≈ 400 MB	180 k/s
Goroutinen	**310**	**22** MB	**495 k/s**

Die Goroutinen erreichen fast den Durchsatz der Ereignisschleife bei der Hälfte der Zeilen. Das ist der eigentliche Gewinn: Man schreibt den Ablauf hintereinander und bekommt die Leistung der zerrissenen Bauform.

Der Vergleich beim Lesen.

Frage	Ereignis-schleife	Goroutinen
Wo geht es weiter?	**Rückruf suchen**	**nächste Zeile**
Zustand je Gerät	eigene Struktur	lokale Variablen
Verstehenszeit	**175** s	**48** s

Faktor dreieinhalb bei der Verstehenszeit — gemessen wie in Band 20, Kapitel 8. Der Grund steht in der zweiten Zeile: Bei Goroutinen ist der Zustand einer Verbindung in lokalen Variablen, nicht in einer von Hand geführten Struktur.

(b) Die Zahl begrenzen.
Das Problem. Eine Goroutine je eingehender Nachricht — bei einer Lastspitze Millionen.

```
sem := make(chan struct{}, 64)   // hoechstens 64 zugleich

for m := range eingang {
sem <- struct{}{}
go func(m Messwert) {
defer func() { <-sem }()
verarbeite(m)
}(m)
}
```

Grenze	Speicher	Durchsatz	Wartezeit
Keine	**unbegrenzt**	bricht ein	∞
8	gering	180 k/s	44 ms
64	≈ 3 MB	**495 k/s**	**13 ms**
512	24 MB	501 k/s	19 ms

Die erste Zeile ist der Fehler, den Goroutinen so leicht machen: Weil sie billig sind, startet man beliebig viele — und bei einer Lastspitze belegen sie mehr Speicher, als da ist. Die Begrenzung kostet fünf Zeilen und macht das System belastbar.

Bemerkenswert ist die letzte Zeile: Mehr Nebenläufigkeit bringt keinen Durchsatz mehr und erhöht die Wartezeit. Das Optimum liegt dort, wo alle Kerne beschäftigt sind und die Warteschlangen kurz bleiben — gemessen, nicht geraten.

(c) Das Fadenmodell darunter.

Goroutinen	Viele, von der Laufzeitumgebung verwaltet
Systemfäden	Wenige — etwa so viele wie Kerne
Die Zuordnung	**Die Laufzeitumgebung verteilt Goroutinen auf Fäden**

Vorgang	**Faden blockiert?**	
Kanaloperation	**nein**	**Goroutine wechselt**
Netzarbeit	nein	eigener Melder
Dateizugriff	teils	
Aufruf nach C	**ja**	**Band 21, Kap. 14**
Enge Rechenschleife	unterbrechbar	seit Go 1.14

Die vierte Zeile ist die Warnung aus Band 21: Ein blockierender C-Aufruf hält den Systemfaden an. Bei wenigen Aufrufen erzeugt die Laufzeitumgebung einen neuen Faden; bei vielen bricht die Nebenläufigkeit zusammen.

Die letzte Zeile war lange ein echtes Problem: Eine Rechenschleife ohne Funktionsaufruf konnte nicht unterbrochen werden und blockierte einen Kern. Seit Go 1.14 unterbricht die Laufzeitumgebung von sich aus — eine der wenigen stillen Verbesserungen, die man bemerkt.

Billig ist nicht kostenlos

Zahl	**Speicher**	**Bewertung**
10^3	≈ 4 MB	unbedenklich
10^5	≈ 400 MB	spürbar
10^6	**≈ 4 GB**	**Grenze**
Nie endend	wächst	**Leck**

Die letzte Zeile ist die gefährlichste und Gegenstand von Kapitel 6: Eine Goroutine, die auf einen Kanal wartet, den niemand mehr bedient, bleibt für immer. Sie verbraucht Speicher und erscheint in keiner Statistik über Speicherlecks.

Pike[18] trennt Nebenläufigkeit von Parallelität in einem Vortrag, der die Verwechslung beendet; Hoare[13] legte 1978 die theoretische Grundlage; die Vorträge zu Nebenläufigkeitsmustern[21] zeigen die wiederkehrenden Bauformen.

Kapitel 5

Kanäle

Der Leitsatz

Nicht	Speicher durch gemeinsamen Zugriff teilen
Sondern	**Speicher teilen, indem man ihn weiterreicht**
Der Unterschied	Ein Wert hat zu jeder Zeit genau einen Besitzer

Die dritte Zeile ist derselbe Begriff wie in Band 21, Kapitel 3 — nur wird er hier nicht vereinbart, sondern durch die Bauform nahegelegt. Wer einen Wert über einen Kanal schickt, gibt ihn ab; wer ihn behält und zugleich verschickt, hat einen Wettlauf.

Die zwei Arten.

	Ohne Puffer	**Mit Puffer**
Senden blockiert	**bis empfangen**	bis Puffer voll
Wirkung	**Gleichschritt**	**Entkopplung**
Gegendruck	**sofort**	verzögert
Verwendung	Übergabe, Signal	Lastspitzen

Die dritte Zeile ist die wichtigste für die Übertragungsschicht: Ein Kanal ohne Puffer bremst den Sender genau dann, wenn der Empfänger nicht nachkommt. Das ist Gegendruck ohne eine einzige Zeile Code — und die Eigenschaft, an der ein System unter Last steht oder fällt.

Wie groß der Puffer sein soll

Größe	Wirkung	
0	**voller Gegendruck**	**Vorgabe**
Klein (8–64)	glättet Schwankungen	
Groß (10^5)	**verbirgt das Problem**	Speicher, Verzögerung
Unbegrenzt	gibt es nicht	aus gutem Grund

Die dritte Zeile beschreibt den häufigsten Fehler: Ein großer Puffer lässt ein überlastetes System eine Weile gesund aussehen und macht den Zusammenbruch später und schwerer. Ein Puffer glättet Schwankungen; er ersetzt keine Kapazität.

Die letzte Zeile ist eine bewusste Auslassung: Go bietet keinen unbegrenzten Kanal, weil er das Gegendrucksignal zerstört — man würde den Speicher füllen, statt den Sender zu bremsen.

`select` und die Regeln

```
select {
case m := <-eingang:
verarbeite(m)
case <-zeit.After(5 * time.Second):
return ErrZeitueberschreitung
case <-ctx.Done():
return ctx.Err()
}
```

Regel	Verhalten	
Mehrere bereit	**zufällig einer**	keine Bevorzugung
Keiner bereit	blockiert	
Mit `default`	**nie blockierend**	Vorsicht
Auf `nil`-Kanal	**nie bereit**	**nützlich**

Die erste Zeile verhindert Aushungern: Wenn zwei Fälle bereit sind, wird gewürfelt statt der erste genommen. Die letzte Zeile ist ein Kunstgriff — einen Kanal auf `nil` zu setzen schaltet seinen Zweig ab, ohne den `select` umzubauen.

Der dritte Fall ist die häufigste Falle: Ein `select` mit `default` in einer Schleife wartet nie und verbraucht einen Kern vollständig — eine beschäftigte Warteschleife, die keine sein sollte.

Wer schließt den Kanal?

Handlung	Erlaubt?	
Sender schließt	**ja**	**die Regel**
Empfänger schließt	**nein**	Sender stürzt ab
Zweimal schließen	**nein**	`panic`
Auf geschlossenen senden	nein	`panic`
Von geschlossenem lesen	ja	Nullwert, `ok=false`

Die Asymmetrie ist der Kern: Schließen ist eine Aussage des Senders — "es kommt nichts mehr". Der Empfänger kann diese Aussage nicht treffen. Bei mehreren Sendern braucht es deshalb einen eigenen Mechanismus, meist eine Zählgruppe.

Übungsaufgaben

(a) Bauen Sie eine Verarbeitungskette.

(b) Messen Sie die Wirkung des Gegendrucks.

(c) Bestimmen Sie, wann Sperren besser sind.

Lösung

(a) Eine Verarbeitungskette.
Die Aufgabe. Messwerte lesen, umrechnen, prüfen, bündeln, senden.

```
func rechne(ctx context.Context, ein <-chan Rohwert)
<-chan Messwert {
aus := make(chan Messwert)
go func() {
defer close(aus)        // Sender schliesst
for r := range ein {
select {
case aus <- umrechne(r):
case <-ctx.Done():  return
}
}
}()
return aus
}
```

Vier Eigenschaften, die zusammengehören: Die Funktion erzeugt ihren Ausgangskanal, schließt ihn selbst, beachtet den Abbruch und gibt einen Nur-Lese-Kanal zurück. Wer alle vier einhält, kann Stufen beliebig verketten, ohne über Lebensdauern nachzudenken.

Die Kette.

```
roh := lies(ctx, geraet)
mess := rechne(ctx, roh)
gut := pruefe(ctx, mess)
pakete := buendle(ctx, gut, 500)
err := sende(ctx, pakete, ziel)
```

Eigenschaft	Ergebnis	
Stufen einzeln prüfbar	**ja**	**Kanal hinein, Kanal heraus**
Reihenfolge änderbar	ja	
Abbruch wirkt überall	**ja**	ein Kontext
Zwischenpuffer	je Stufe wählbar	

Die erste Zeile ist der Prüfgewinn: Jede Stufe ist eine reine Abbildung von Kanal auf Kanal und lässt sich mit einer Liste von Eingaben prüfen — ohne Gerät, ohne Netz, ohne Attrappe. Das ist die Trennung aus Band 20, Kapitel 6 in nebenläufiger Form.

(b) Die Wirkung des Gegendrucks.
Der Versuch. Der Empfänger wird langsamer als die Sender.

Puffer	Speicher	Verlust	Verzögerung
0	**2** MB	**0**	**12** ms
1000	6 MB	0	310 ms
10^5	140 MB	0	**31 s**
10^6	**1,4 GB**	**Absturz**	—

Die dritte Zeile zeigt den eigentlichen Schaden: Kein Verlust, aber einunddreißig Sekunden Verzögerung — ein Messwert erreicht die Klinik eine halbe Minute zu spät. Bei einer Grenzwertwarnung ist das schlimmer als ein sauberer Verlust mit Meldung.

Die erste Zeile ist die richtige Wahl: Ohne Puffer bremst der Gegendruck bis zum Gerät durch, und das Gerät kann selbst entscheiden, was es verwirft. Die Entscheidung fällt dort, wo man sie fachlich treffen kann.

(c) Wann Sperren besser sind.

Lage	Kanal?	Sperre?
Werte weiterreichen	**ja**	nein
Ablauf koordinieren	ja	nein
Zwischenspeicher schützen	nein	**ja**
Zähler erhöhen	nein	atomar
Konfiguration lesen	nein	Lesesperre

Die dritte Zeile widerspricht dem Leitsatz und ist trotzdem richtig: Für einen Zwischenspeicher, auf den viele lesend zugreifen, ist eine Sperre einfacher und schneller als eine Goroutine, die ihn verwaltet. Der Leitsatz ist eine Voreinstellung, kein Verbot.

Die Messung.

Vorgang	Kanal	Sperre	
Zähler erhöhen	260 ns	**18** ns	14×
Wert übergeben	**95** ns	110 ns	
Zwischenspeicher lesen	240 ns	**6** ns	40×

Ein Kanal kostet etwa hundert Nanosekunden — viel für einen Zähler, wenig für eine Netzübertragung. Die Wahl folgt aus der Häufigkeit: Was millionenfach je Sekunde geschieht, braucht keine Nachrichten.

Kanäle lösen keine Entwurfsfehler

Erwartung	Erfüllt?	
Keine Wettläufe mehr	**nein**	**wenn man Zeiger schickt**
Keine Verklemmungen	**nein**	Kap. 6
Automatisch schneller	nein	
Übersichtlicher Ablauf	**ja**	der eigentliche Gewinn

Die erste Zeile ist die verbreitetste Fehlannahme: Wer einen Zeiger über einen Kanal schickt und ihn danach weiter benutzt, hat denselben Wettlauf wie mit gemeinsamem Speicher. Der Kanal überträgt den Besitz nur, wenn man ihn tatsächlich abgibt.

Hoare[13] beschrieb die kommunizierenden Prozesse, aus denen die Kanäle stammen; die Mustervorträge[21] zeigen die wiederkehrenden Bauformen; das Speichermodell[10] legt fest, welche Sichtbarkeitszusicherungen daraus folgen.

Kapitel 6

Wo es schiefgeht

Drei Fehlerklassen

Klasse	Symptom	Findet
Wettlauf	falsche Werte, selten	**Wettlaufprüfer**
Verklemmung	**alles steht**	Laufzeit-umgebung
Goroutinenleck	**Speicher wächst**	Profil, mühsam

Die dritte Klasse gibt es in C nicht, weil dort niemand hunderttausend Fäden startet. Sie ist der Preis dafür, dass Goroutinen billig sind — und die einzige der drei, für die es kein zuverlässiges Werkzeug gibt.

Die Verklemmung, die erkannt wird.

Lage	Erkannt?	
Alle Goroutinen warten	**ja**	**Programm bricht ab**
Einige warten, andere laufen	**nein**	sieht gesund aus
Warten auf Netz	nein	zählt als Arbeit
Warten auf Sperre	nein	

Nur die vollständige Verklemmung wird gemeldet, und die ist der harmlose Fall — sie fällt beim ersten Lauf auf. Die zweite Zeile ist die gefährliche: Eine Teilverklemmung sieht von außen aus wie ein System, das gerade nichts zu tun hat.

Der Wettlaufprüfer

Eigenschaft	Wert	
Laufzeit	×5–10	
Speicher	×5–10	
Fehlalarme	**keine**	**meldet nur Echtes**
Übersieht	nicht ausgeführte Pfade	Band 21, Kap. 10

Die dritte Zeile ist die entscheidende Eigenschaft: Der Prüfer meldet keine Fehlalarme — was er meldet, ist ein Wettlauf. Damit entfällt die Abwägung aus Band 21, Kapitel 10, und jede Meldung wird ernst genommen.

Der Preis steht in der letzten Zeile: Er sieht nur, was ausgeführt wurde. Ein Wettlauf in einem Fehlerpfad, den die Prüfung nie erreicht, bleibt unentdeckt — deshalb gehört der Prüfer über den vollständigen Prüfbestand, nicht über ein Beispiel.

Das Goroutinenleck

Der häufigste Fall.

```
func hole(urls []string) string {
aus := make(chan string)    // ohne Puffer!
for _, u := range urls {
go func(u string) { aus <- lade(u) }(u)
}
return <-aus                // nur einer wird gelesen
}
```

Bei n Adressen	Ergebnis	
Gelesen	1	
Beendet	**1**	
Blockiert für immer	**$n-1$**	**das Leck**
Speicher je Aufruf	$(n-1)\cdot$Keller	wächst

Sieben Zeilen, funktioniert scheinbar, und bei jedem Aufruf bleiben $n-1$ Goroutinen für immer stehen. Nach einem Tag Betrieb sind das Millionen — und in keiner Speicherstatistik erscheint ein Leck, weil formal alles erreichbar ist.

Die Abhilfe: drei Möglichkeiten.

Weg	Aufwand	
Puffer der Größe n	**ein Zeichen**	**alle senden, alle enden**
Abbruchkontext	gering	sauberer
Alle Ergebnisse lesen	gering	wenn man sie braucht

Die erste Zeile ist ein Lehrstück: `make(chan string, len(urls))` — ein Argument mehr, und die Fehlerklasse verschwindet. Solche Ein-Zeichen-Unterschiede zwischen richtig und falsch sind der Grund für Kapitel 10.

Warum das Leck so schwer zu finden ist

Eigenschaft	Wert	
Meldung	**keine**	
Absturz	erst nach Tagen	**Speicher voll**
Wettlaufprüfer	**sieht nichts**	kein Wettlauf
Verklemmungs-melder	nein	andere laufen
Goroutinenprofil	**ja**	**das einzige Mittel**

Vier Werkzeuge schweigen, eines hilft. Das Goroutinenprofil zeigt, wie viele Goroutinen an welcher Zeile warten — und eine Zeile mit zweihunderttausend wartenden Goroutinen ist ein eindeutiger Befund.

Daraus folgt eine Betriebsregel: Die Zahl der Goroutinen gehört überwacht, wie der Speicherverbrauch. Sie ist die einzige Kennzahl, die diese Fehlerklasse früh sichtbar macht.

Übungsaufgaben

(a) Verfolgen Sie einen Wettlauf.

(b) Bauen Sie eine Verklemmung nach.

(c) Prüfen Sie auf Goroutinenlecks.

Lösung

(a) Einen Wettlauf verfolgen.

```
var zaehler int              // ungeschuetzt

for i := 0; i < 1000; i++ {
go func() { zaehler++ }()
}
```

Lauf	Ergebnis	Erwartet	
1	947	1000	falsch
2	1000	1000	zufällig richtig
3	982	1000	falsch
Mit Prüfer	—	—	**gemeldet**

Die zweite Zeile ist die gefährliche: Manchmal stimmt das Ergebnis. Ein Wettlauf, der in neun von zehn Läufen richtig rechnet, überlebt jede Prüfung, die ihn nicht gezielt sucht — und der Prüfer meldet ihn beim ersten Lauf.

Abhilfe	Dauer	
`sync.Mutex`	18 ns	
`atomic.AddInt64`	**6** ns	**für Zähler**
Kanal	260 ns	unpassend
Je Goroutine zählen	**0,4** ns	**am Ende addieren**

Die letzte Zeile ist die beste und wird selten gewählt: Wenn jede Goroutine ihren eigenen Zähler führt und erst am Ende addiert wird, gibt es keinen gemeinsamen Zustand — die Fehlerklasse verschwindet durch die Bauform, nicht durch Schutz.

(b) Eine Verklemmung nachbauen.

```
// Goroutine A     // Goroutine B
a.Lock()           b.Lock()
b.Lock()           a.Lock()
```

Bedingung	Erfüllt?	
Gegenseitiger Ausschluss	ja	
Halten und Warten	ja	
Kein Entzug	ja	
Zirkuläres Warten	**ja**	**hier ansetzen**

Coffmans vier Bedingungen aus Band 4 — alle vier müssen erfüllt sein. Die vierte ist die einzige, die man in der Praxis billig beseitigt: Wenn alle Sperren in derselben Reihenfolge genommen werden, kann kein Zyklus entstehen.

Die Regel lautet deshalb: eine feste, aufgeschriebene Sperrreihenfolge — und sie gehört in die Kopfkommentare des Pakets, nicht in den Kopf des Entwicklers.

(c) Auf Goroutinenlecks prüfen.

```
func TestKeinLeck(t *testing.T) {
vorher := runtime.NumGoroutine()
hole(adressen)
time.Sleep(100 * time.Millisecond)
if n := runtime.NumGoroutine(); n > vorher {
t.Errorf("%d Goroutinen uebrig", n-vorher)
}
}
```

Eigenschaft	Bewertung	
Aufwand	8 Zeilen	
Findet das Beispiel	**ja**	
Zuverlässig	**Wartezeit nötig**	sprödes Element
Als Bibliothek	vorhanden	besser

Die dritte Zeile ist die Schwäche: Die Wartezeit ist geraten. Zu kurz, und die Prüfung meldet falsche Lecks; zu lang, und der Prüflauf wird langsam. Fertige Bibliotheken lösen das mit wiederholtem Nachsehen statt einer festen Pause.

Trotz der Schwäche lohnt die Prüfung: Acht Zeilen je Paket finden die Fehlerklasse, für die es sonst kein Werkzeug gibt — und sie laufen bei jeder Übergabe mit.

Die Reihenfolge der Maßnahmen

1	**Keinen gemeinsamen Zustand** — dann gibt es nichts zu schützen
2	Wenn nötig, Werte über Kanäle weiterreichen
3	Wenn das unpassend ist, atomare Operationen
4	**Wenn das nicht reicht, Sperren — mit fester Reihenfolge**
5	**Immer: Wettlaufprüfer in der Prüfstrecke**

Die Reihenfolge ist nach Fehleranfälligkeit geordnet, nicht nach Eleganz. Der erste Schritt löst mehr Probleme als alle anderen zusammen — und wird am seltensten erwogen, weil er eine Entwurfsänderung verlangt statt eines Sprachmittels.

Der Wettlaufprüfer[28] findet die erste Klasse zuverlässig; das Speichermodell[10] legt fest, was überhaupt ein Wettlauf ist; Lamport[15] begründete den Begriff der Ordnung von Ereignissen, auf dem das beruht.

Teil III

Der Entwurf

Kapitel 7

Pakete

Sichtbarkeit mit einem Zeichen

Name	Sichtbar	
`Messwert`	**überall**	**großer Anfangsbuchstabe**
`messwert`	nur im Paket	
Zwischenstufen	gibt es nicht	kein `protected`

Zwei Stufen statt vier, und die Entscheidung ist im Namen sichtbar — man muss keine Deklaration suchen, um zu wissen, ob etwas öffentlich ist. Der Preis: Es gibt kein "sichtbar für verwandte Pakete", und manchmal fehlt genau das.

Wie groß ein Paket sein soll.

Maßstab	Trägt?	
Zeilenzahl	**nein**	Band 20, Kap. 5
Eine Klasse je Datei	nein	**fremde Gewohnheit**
Ein Geheimnis	**ja**	**Parnas**
Was gemeinsam geändert wird	ja	

Ein Paket ist die Einheit der Kapselung, nicht die Datei. Innerhalb eines Pakets darf alles auf alles zugreifen — die Grenze verläuft am Paketrand. Deshalb ist die Frage nicht "wie viele Dateien", sondern "was verbirgt dieses Paket".

Der Paketname ist Teil des Aufrufs

Aufruf	Bewertung	
`uebertragung.Uebertrage`	doppelt	Stotterei
`uebertragung.Starte`	**gut**	
`util.Hilfsfunktion`	**nichtssagend**	**kein Geheimnis**
`ring.Neu`	gut	

Der Paketname steht bei jedem Aufruf davor — deshalb wiederholt man ihn nicht im Funktionsnamen. Die dritte Zeile ist der häufigste Entwurfsfehler: Ein Paket namens `util` verbirgt kein Geheimnis, sondern sammelt Reste. Es wächst, bis es von allem abhängt.

Abhängigkeiten ohne Zyklen

Eigenschaft	In Go	
Zirkuläre Einschlüsse	**Fehler**	**beim Übersetzen**
Unbenutzte Einschlüsse	Fehler	
Bauzeit	**linear**	Band 21, Kap. 11
Abhängigkeitsgraph	ohne Zyklen	erzwungen

Die erste Zeile ist eine harte Sprachregel und wirkt als Entwurfszwang: Wenn zwei Pakete einander brauchen, gehören sie entweder zusammen oder es fehlt ein drittes. Der Übersetzer zwingt zur Klärung, die man sonst jahrelang vertagt.

Der übliche Fall und seine Auflösung.

Lage	Auflösung	
`geraet` braucht `uebertragung`	**Schnittstelle beim Aufrufer**	Kap. 3
Beide brauchen Typen	drittes Paket	nur Typen
Wirklich untrennbar	**zusammenlegen**	

Die erste Zeile ist der häufigste Fall und wird durch die Gestaltprüfung gelöst: Das Übertragungspaket definiert die Schnittstelle, die es braucht — und das Gerätepaket erfüllt sie, ohne es zu wissen. Der Zyklus verschwindet, ohne dass jemand etwas anpassen muss.

Fremde Abhängigkeiten

Frage	Vor der Aufnahme	
Wie viele Zeilen?	**prüfen**	oft mehr als erwartet
Wie viele weitere?	**prüfen**	**der Baum wächst**
Wer pflegt es?	prüfen	
Selbst schreiben?	**oft billiger**	bei < 200 Zeilen

Die zweite Zeile ist die unterschätzte: Eine Bibliothek mit dreihundert Zeilen bringt vierzig weitere mit, und jede davon ist Code, den man ausliefert und für den man haftet. In einem Medizinprodukt gehört jede in die Stückliste.

Die letzte Zeile ist eine Go-Eigenheit mit gutem Grund: Weil die Standardbibliothek viel abdeckt, ist "selbst schreiben" öfter richtig als anderswo — und zweihundert Zeilen eigener Code sind leichter zu prüfen als eine fremde Abhängigkeitskette.

Übungsaufgaben

(a) Schneiden Sie die Übertragungsschicht in Pakete.

(b) Lösen Sie einen Zyklus auf.

(c) Bewerten Sie eine fremde Abhängigkeit.

Lösung

(a) Die Übertragungsschicht schneiden.

Paket	Geheimnis	Zeilen	Hängt von
`messwert`	**Datenmodell**	180	**0**
`geraet`	Protokoll des Geräts	640	1
`ring`	**Pufferverwaltung**	210	1
`uebertragung`	Ablauf, Wiederholung	520	2
`ziel`	Netzprotokoll	380	1
`main`	Zusammenbau	90	5

Die erste Zeile ist die wichtigste: Das Datenmodell hängt von nichts ab. Damit können alle anderen es benutzen, ohne einander zu kennen — Wirths Rangordnung aus Band 20 als Paketstruktur.

Die letzte Zeile ist die zweite Regel: Nur `main` kennt alle. Der Zusammenbau geschieht an einer Stelle, und die ist neunzig Zeilen lang. Wer diese Regel einhält, kann jedes andere Paket einzeln prüfen und ersetzen.

(b) Einen Zyklus auflösen.
Die Lage. `geraet` will Fehler an `uebertragung` melden.

Weg	Zyklus?	
`geraet` ruft `uebertragung`	**ja**	Übersetzungsfehler
Rückrufadresse	nein	**Funktionswert**
Fehler zurückgeben	**nein**	**am einfachsten**
Drittes Paket	nein	mehr Struktur

Die dritte Zeile ist fast immer die richtige und wird oft übersehen: Statt zu melden, gibt man den Fehler zurück und lässt den Aufrufer entscheiden. Der Zyklus verschwindet, weil die Abhängigkeit nur noch in eine Richtung zeigt.

Das ist derselbe Gedanke wie in Band 21, Kapitel 11: Abhängigkeiten zeigen nach unten. Der Unterschied ist, dass Go es erzwingt statt zu empfehlen — und dass die Klärung damit sofort stattfindet statt nach fünf Jahren.

(c) Eine fremde Abhängigkeit bewerten.
Der Fall. Eine Bibliothek für strukturierte Protokollierung.

	Bibliothek	Standard	Selbst
Eigene Zeilen	**0**	**0**	140
Fremde Zeilen	≈ 24 000	**0**	0
Weitere Pakete	**11**	0	0
Leistung	sehr gut	gut	ausreichend
In der Stückliste	12 Einträge	**0**	0

Seit Go 1.21 gibt es strukturierte Protokollierung in der Standardbibliothek — damit wird die mittlere Spalte fast immer richtig. Vor 2023 war die Entscheidung schwerer, und viele haben die vierundzwanzigtausend Zeilen genommen.

Die vorletzte Zeile zeigt die Größenordnung des Unterschieds: sehr gut gegen gut. Bei einer Übertragungsschicht, die auf das Netz wartet, ist das nicht messbar — und elf zusätzliche Pakete sind es sehr wohl.

Die Standardbibliothek zuerst

Bereich	Standard reicht?	
Netz, HTTP	**ja**	**ungewöhnlich vollständig**
Verschlüsselung	ja	
Protokollierung	ja	seit 1.21
Datenbanktreiber	nein	Schnittstelle ja
Zeitzonen, Text	ja	

Die erste Zeile ist ungewöhnlich: Ein vollständiger HTTP-Bedienteil und -Anfrager in der Standardbibliothek, betriebstauglich, ohne Rahmenwerk. In den meisten Sprachen ist das der Ort, an dem man die erste große Abhängigkeit aufnimmt.

Daraus folgt die praktische Regel dieses Kapitels: Erst nachsehen, ob es die Standardbibliothek kann. Bei Go lautet die Antwort öfter ja als anderswo — und jede vermiedene Abhängigkeit ist Code, den man nicht ausliefert und nicht verantwortet.

Parnas[17] begründete das Geheimnisprinzip, an dem sich der Paketschnitt misst; “Effective Go”[7] legt die Namensregeln fest; Cox[3] beschreibt, was fremde Abhängigkeiten tatsächlich kosten.

Kapitel 8

Zusammensetzung

Einbetten statt erben

```
type Basis struct { id ID; erzeugt time.Time }
func (b Basis) ID() ID { return b.id }

type Laborbefund struct {
Basis            // eingebettet, ohne Feldnamen
Wert Ergebnis
}

b := Laborbefund{...}
b.ID()                // Methode von Basis, hochgereicht
```

	Vererbung	Einbettung
Beziehung	**ist ein**	**hat ein**
Methode überschreiben	ja, dynamisch	**nur verdecken**
Basis ruft abgeleitet	ja	**nein**
Mehrfach	schwierig	**beliebig**

Die dritte Zeile ist der entscheidende Unterschied und der häufigste Stolperstein für Umsteiger: Eine eingebettete Basis weiß nichts von dem Typ, der sie enthält. Es gibt keine Schablonenmethode, kein Aufrufen nach unten — und damit auch keine der Überraschungen, die daraus entstehen.

Was das Verdecken bedeutet.

Aufruf	Ruft	
`b.Beschreibung()`	**die äußere**	verdeckt
`b.Basis.Beschreibung()`	die innere	ausdrücklich
Aus `Basis` heraus	**immer die innere**	**nie die äußere**

Die dritte Zeile ist der Punkt, an dem Rahmenwerksmuster scheitern. Wer aus einer Basismethode heraus eine überschriebene Methode aufrufen will, kann das nicht — er muss die Abhängigkeit ausdrücklich übergeben, meist als Schnittstelle.

Die Umkehrung, die daraus folgt

```
// statt:  Basis ruft ueberschriebene Methode
type Formatierer interface { Beschreibe(Basis) string }

func (b Basis) Text(f Formatierer) string {
return f.Beschreibe(b)
}
```

Was in einer Klassenhierarchie implizit über die Vererbung liefe, wird hier ein Argument. Das ist mehr Schreibarbeit und beantwortet beim Lesen eine Frage, die man sonst suchen muss: Welche Umsetzung wird aufgerufen? Die, die dasteht.

Wo die Einbettung nützt

Zweck	Passend?	
Gemeinsame Felder	ja	**Kennung, Zeitstempel**
Schnittstelle weiterreichen	ja	**Verhalten ergänzen**
Sperre in eine Struktur	umstritten	macht sie öffentlich
Hierarchie nachbauen	nein	**funktioniert nicht**

Die zweite Zeile ist die eleganteste Anwendung: Man bettet eine Schnittstelle ein, überschreibt eine Methode und reicht den Rest durch. Damit ergänzt man fremdes Verhalten, ohne den fremden Typ zu kennen — in fünf Zeilen.

```
type ZaehlenderSchreiber struct {
io.Writer           // Schnittstelle eingebettet
N int64
}

func (z *ZaehlenderSchreiber) Write(p []byte) (int, error)
{
n, err := z.Writer.Write(p)
z.N += int64(n)
return n, err
}
```

Zehn Zeilen, und jeder Empfänger von Bytes bekommt einen Zähler — Datei, Netzverbindung, Komprimierer. Das ist das Dekorierermuster aus Band 17 ohne jedes Gerüst.

Die Falle der eingebetteten Sperre

```
type Zwischenspeicher struct {
sync.Mutex          // eingebettet:  Lock() ist
oeffentlich!
daten map[ID]Messwert
}
```

Folge	Bewertung	
`Lock` von außen aufrufbar	**ja**	**Teil der Schnittstelle**
Sperrstrategie änderbar	nein	**Aufrufer hängen daran**
Als Feld benannt	**besser**	`mu sync.Mutex`

Die Einbettung macht `Lock` und `Unlock` zu öffentlichen Methoden des Zwischenspeichers — also zu einem Versprechen, das man nicht zurücknehmen kann. Ein benanntes Feld `mu` ist ein Zeichen mehr und hält die Sperrstrategie im Paket.

Übungsaufgaben

(a) Bauen Sie eine Hierarchie um.

(b) Messen Sie den Unterschied beim Lesen.

(c) Bestimmen Sie, wo Wiederholung besser ist.

Lösung

(a) Eine Hierarchie umbauen.
Der Ausgangszustand. Fünf Befundarten, drei Ebenen, vier Schablonenmethoden.

Element	Vorher	Nachher
Typen	8	**5**
Ebenen	3	**1**
Schnittstellen	1 groß	**3** klein
Zeilen	520	610
Doppelter Code	0	≈ 70 Zeilen

Neunzig Zeilen mehr, davon siebzig Wiederholung — das ist der Preis, ehrlich beziffert. Wer nur diese Tabelle sieht, hält den Umbau für eine Verschlechterung.

Was sich dafür ändert.

Vorgang	Vorher	Nachher	
Neue Befundart	Hierarchie prüfen	**neuer Typ**	
Zwei Eigenschaften	**unmöglich**	ja	
Eine Art einzeln prüfen	Basis nötig	**allein**	
Verstehenszeit	195 s	**62** s	3,1×

Faktor drei bei der Verstehenszeit, gemessen wie in Band 20. Die zweite Zeile war der Anlass des Umbaus — ein Befund mit Bild und Zahlen ließ sich in der Hierarchie nicht darstellen, und alle Umgehungen waren schlechter als die Wiederholung.

(b) Der Unterschied beim Lesen.

Frage	Hierarchie	Zusammen-setzung
Welche Methode läuft?	**84** s	**9** s
Was tut die Basis?	41 s	12 s
Wo wird der Wert gesetzt?	52 s	18 s
Was ändert sich bei X?	**118** s	**31** s

Die erste Zeile trägt den Unterschied: In einer Hierarchie mit drei Ebenen muss man alle drei ansehen, um zu wissen, welche Umsetzung läuft. Bei Zusammensetzung steht sie beim Typ — neun Sekunden statt vierundachtzig.

Das ist die Rechnung, die den Preis rechtfertigt: siebzig Zeilen Wiederholung gegen dreiviertel weniger Verstehenszeit — bei einer Tätigkeit, die dreißig Prozent der Arbeitszeit ausmacht.

(c) Wo Wiederholung besser ist.

Lage	Zusammenfassen?	
Zufällig gleich	**nein**	**Band 20, Kap. 14**
Ändert sich aus demselben Grund	**ja**	
Drei oder mehr Fälle	ja	Regel der drei
Zwei Fälle	noch nicht	
Fachlich verschieden	**nein**	trotz gleichem Code

Die letzte Zeile ist die wichtigste und in Go besonders sichtbar: Zwei Prüffunktionen, die heute gleich aussehen — eine für Laborwerte, eine für Abrechnungsbeträge — werden sich auseinanderentwickeln. Wer sie zusammenlegt, muss sie später mühsam trennen.

Go begünstigt diese Zurückhaltung, weil es das Zusammenlegen umständlicher macht. Ob das eine Tugend oder ein Mangel ist, hängt davon ab, wie oft man die Trennung später gebraucht hätte.

Die Umstellung kostet Wochen

Herkunft	Umgewöhnung	
Java, C#	**schwer**	**Hierarchien fehlen**
C++	schwer	
Python	mittel	
C	**leicht**	gab es dort auch nicht

Der typische Verlauf: In den ersten Wochen baut man Hierarchien mit Einbettung nach, es funktioniert nicht richtig, und man ärgert sich über die Sprache. Nach ein bis zwei Monaten hört man auf, es zu versuchen — und findet die Lösungen einfacher.

Das ist keine Aussage darüber, welcher Ansatz besser ist. Es ist eine Warnung an den, der Go beurteilen will: Das Urteil aus den ersten Wochen misst die Umgewöhnung, nicht die Sprache.

Ousterhout[16] begründet, warum tiefe Schnittstellen wichtiger sind als Hierarchien; Donovan und Kernighan[6] zeigen Einbettung mit ihren Grenzen; Bodner[1] behandelt die Umge-

wöhnung ausdrücklich.

Kapitel 9

Der Nullwert

Jeder Typ hat einen brauchbaren Anfangswert

Typ	Nullwert	Brauchbar?
Zahl	0	ja
Zeichenkette	`""`	ja
`sync.Mutex`	**entsperrt**	**sofort verwendbar**
`bytes.Buffer`	leer	**sofort verwendbar**
Abschnitt	`nil`	anhängbar, zählbar
Abbildung	`nil`	**lesbar, nicht schreibbar**
Zeiger	`nil`	Absturz beim Zugriff

Die dritte und vierte Zeile sind der Entwurfsgedanke: Eine Sperre und ein Puffer sind ohne Erzeugerfunktion einsatzbereit. Deshalb braucht Go keine Konstruktoren — der Nullwert ist der gültige Anfangszustand.

Was das für eigene Typen bedeutet.

Die Regel	Entwirf den Typ so, dass sein Nullwert brauchbar ist
Der Gewinn	**Keine Erzeugerfunktion, kein Vergessen**
Wenn unmöglich	Eine `Neu`-Funktion — und dann ausnahmslos

Die dritte Zeile ist die ehrliche Einschränkung: Nicht jeder Typ hat einen sinnvollen Nullzustand. Ein Ring mit Kapazität null ist unbrauchbar. Dann braucht es eine Erzeugerfunktion — und die Regel ist, dass es keinen zweiten Weg gibt, den Typ anzulegen.

Die Falle: Abbildung gegen Abschnitt

```
var s []int
s = append(s, 1)          // funktioniert
len(s)                    // funktioniert

var m map[string]int
x := m["a"]               // funktioniert, liefert 0
m["a"] = 1                // panic!
```

Zwei ähnlich aussehende Typen, zwei verschiedene Regeln. Ein leerer Abschnitt ist voll benutzbar, eine leere Abbildung nur lesbar. Das ist eine Unstimmigkeit im Entwurf der Sprache, und man muss sie schlicht wissen.

Abschnitte: die drei Zahlen

Bestandteil	Bedeutung	
Zeiger	**auf das Feld**	**geteilt!**
Länge	sichtbare Elemente	
Kapazität	**belegter Platz**	bestimmt `append`

Ein Abschnitt ist ein Fenster auf ein Feld, kein Feld. Zwei Abschnitte können dasselbe Feld zeigen — und dann wirkt eine Änderung im einen im anderen. Das ist die Fehlerquelle des Kapitels und dieselbe wie in Band 21, Kapitel 3.

Der Fall.

```
alle := make([]Messwert, 0, 100)
alle = append(alle, werte...)    // 60 Elemente

erste := alle[:10]
erste = append(erste, neu)       // ueberschreibt alle[10]!
```

	Länge	Kapazität	
`alle`	60	100	
`erste`	10	**100**	**geerbt**
Nach `append`	11	100	**schreibt in `alle`**

Der Teilabschnitt erbt die volle Kapazität — deshalb schreibt `append` in das gemeinsame Feld statt ein neues anzulegen. Ein Messwert wird stillschweigend überschrieben, und niemand bemerkt es.

Die Abhilfe: die dritte Zahl

```
erste := alle[:10:10]    // Laenge 10, Kapazitaet 10
erste = append(erste, neu) // legt neu an – sicher
```

Form	Kapazität	
`alle[:10]`	100	**teilt das Feld**
`alle[:10:10]`	**10**	**trennt beim Anhängen**
Kopie	10	trennt sofort

Die dritte Zahl ist wenig bekannt und genau dafür da. Die Regel für Bibliotheken lautet: Wer einen Teilabschnitt nach außen gibt, begrenzt die Kapazität — sonst kann der Empfänger in fremden Speicher schreiben, ohne es zu wollen.

Übungsaufgaben

(a) Entwerfen Sie einen Typ mit brauchbarem Nullwert.

(b) Verfolgen Sie ein Speicherleck über Abschnitte.

(c) Bestimmen Sie, wann eine Erzeugerfunktion nötig ist.

Lösung

(a) Brauchbarer Nullwert.
Der erste Entwurf.

```
type Sammler struct { werte []Messwert; grenze int }

func NeuSammler(grenze int) *Sammler { ...  }
```

Ohne Erzeugerfunktion ist die Grenze null — und ein Sammler, der nichts sammelt, ist unbrauchbar. Man braucht also die Erzeugerfunktion, und jeder muss daran denken.

Der zweite Entwurf.

```
type Sammler struct {
werte []Messwert
Grenze int    // 0 bedeutet:  keine Grenze
}

var s Sammler       // sofort verwendbar
s.Nimm(m)
```

	Erster	Zweiter
Erzeugerfunktion nötig	ja	nein
Als Feld einbettbar	Zeiger nötig	direkt
Vergessbar	ja	nein
Zeilen	14	9

Die zweite Zeile ist der praktische Gewinn: Ein Typ mit brauchbarem Nullwert kann als Feld in einer anderen Struktur stehen, ohne dass deren Erzeugerfunktion ihn anlegen muss. Das pflanzt sich fort — eine Struktur mit fünf solchen Feldern braucht selbst keine.

Der Kunstgriff steht im Kommentar: Null bedeutet "keine Grenze" statt "Grenze null". Man wählt die Bedeutung so, dass der Nullwert sinnvoll ist — eine Entwurfsentscheidung, kein Trick.

(b) Ein Speicherleck über Abschnitte.
Der Fall. Aus einer Datei von 40 MB werden zehn Messwerte behalten.

```
roh, _ := os.ReadFile(pfad)     // 40 MB
kopf := roh[:200]               // scheinbar 200 Byte
merke(kopf)                   // haelt 40 MB fest!
```

Gemerkt	Belegt	
Abschnitt	200 B	40 **MB festgehalten**
Kopie	**200** B	200 **B**
Bei 1000 Dateien	**40 GB**	Absturz

Die Speicherbereinigung gibt ein Feld erst frei, wenn kein Abschnitt mehr darauf zeigt. Zweihundert Byte halten vierzig Megabyte fest — und das ist kein Fehler der Bereinigung, sondern eine Folge davon, dass ein Abschnitt ein Fenster ist.

Die Regel: Wer einen kleinen Teil eines großen Puffers länger behält, kopiert ihn. Drei Zeilen mehr, und der Unterschied ist Faktor zweihunderttausend.

(c) Wann eine Erzeugerfunktion nötig ist.

Lage	Nötig?	
Abbildung im Typ	**ja**	**`nil` nicht schreibbar**
Kanal im Typ	ja	`nil` blockiert ewig
Goroutine zu starten	**ja**	
Nur Zahlen und Abschnitte	nein	Nullwert genügt
Prüfung der Argumente	ja	

Die erste und zweite Zeile sind die häufigsten Gründe: Ein Typ mit einer Abbildung oder einem Kanal darin hat keinen brauchbaren Nullwert. Wer das vermeiden will, legt die Abbildung erst bei der ersten Verwendung an — was wiederum eine Sperre braucht, wenn mehrere Goroutinen zugreifen.

Damit ist die Abwägung benannt: Der brauchbare Nullwert ist ein Ziel, kein Dogma. Wo er Umwege erzwingt, die schwerer zu lesen sind als eine `Neu`-Funktion, nimmt man die `Neu`-Funktion.

Wert oder Zeiger?

Lage	Zeiger?	
Methoden ändern den Wert	**ja**	**sonst wirkungslos**
Struktur ist groß	ja	ab ≈ 64 B
Enthält eine Sperre	**ja**	**Kopie zerstört sie**
Kleiner Wert, unveränderlich	nein	als Wert
Gemischt im selben Typ	**nie**	verwirrend

Die dritte Zeile ist eine harte Falle: Eine Struktur mit eingebetteter Sperre darf nicht kopiert werden — die Kopie hat eine eigene Sperre, und der Schutz ist fort. Der Prüfer `go vet` findet das, und man sollte ihn deshalb laufen lassen.

Die letzte Zeile ist eine Stilregel mit gutem Grund: Ein Typ, dessen Methoden teils Wert- und teils Zeigerempfänger haben, verhält sich unvorhersagbar. Man entscheidet sich einmal je Typ.

Gerrand[9] erklärt Abschnitte mit ihren drei Zahlen und den Folgen; die Spezifikation[11] legt die Nullwerte je Typ fest; Kennedy[14] behandelt die Entwurfsfolgen ausführlich.

Teil IV

Das Handwerk

Kapitel 10

Werkzeuge

Was mitgeliefert wird

Werkzeug	Leistet	Aufwand
`gofmt`	**Formatierung**	keiner
`go vet`	**häufige Fehler**	keiner
`go test`	Prüfungen, Abdeckung	keiner
`-race`	**Wettläufe**	Laufzeit
`pprof`	**Profil**	gering
`go doc`	Dokumentation	keiner

Sechs Werkzeuge, keine Installation, keine Konfiguration — sie sind Teil der Sprachauslieferung. Das ist der praktische Unterschied zu Band 21, wo dieselben Aufgaben eine Werkzeugkette verlangen, die jemand einrichten muss.

Was das für die Verbreitung bedeutet.

Praxis	In C	In Go
Formatierer läuft	$\approx 35\,\%$	$\approx$ **99** %
Statische Prüfung	$\approx 25\,\%$	$\approx$ **85** %
Wettlaufprüfer	$< 10\,\%$	$\approx 60\,\%$

Die Zahlen sind Schätzungen und die Größenordnung ist belastbar: Ein Werkzeug, das mitgeliefert wird und ohne Einrichtung läuft, wird benutzt. Eines, das man aufsetzen muss, oft nicht — die Lehre aus Band 21, Kapitel 10.

Was `go vet` findet

Fehler	Sonst bemerkt?	
Formatzeichen passt nicht	zur Laufzeit	gefunden
Struktur mit Sperre kopiert	**nie**	**gefunden**
Schleifenvariable in Goroutine	selten	gefunden
Fehler nicht geprüft	nie	teils
Unerreichbarer Code	nie	gefunden

Die zweite Zeile ist die wertvollste — eine kopierte Sperre schützt nichts und fällt sonst nie auf. Die dritte war bis Go 1.22 eine der häufigsten Fehlerquellen überhaupt; seither ist sie durch eine Sprachänderung beseitigt.

Das Profil

Art	Zeigt	
Rechenzeit	**wo die Zeit vergeht**	
Speicher	**wer belegt**	Halde
Blockierung	**wo gewartet wird**	Kanäle, Sperren
Goroutinen	**wo sie stehen**	**Kap. 6**
Sperrenstreit	wer wartet auf wen	

Die dritte und vierte Art gibt es in anderen Sprachen selten und sie sind für nebenläufige Programme die wichtigsten: Ein Programm, das nur wartet, verbraucht keine Rechenzeit — ein Rechenzeitprofil zeigt dann nichts, obwohl das Problem offensichtlich ist.

Im Betrieb.

```
import _ "net/http/pprof"

go func() {
log.Println(http.ListenAndServe("localhost:6060", nil))
}()
```

Eigenschaft	Wert	
Zeilen	**4**	
Laufzeitkosten	≈ 0	**bis abgefragt**
Im Betrieb nutzbar	**ja**	Band 21, Kap. 12
Sicherheit	**nur lokal binden**	**nie nach außen**

Vier Zeilen, und man kann ein laufendes Programm im Betrieb befragen — das Mittel, das Band 21 vermisst. Die letzte Zeile ist keine Formalie: Die Profilschnittstelle gibt Speicherinhalte preis und gehört niemals ins offene Netz.

Der Formatierer ersetzt kein Urteil

Werkzeug findet	Werkzeug findet nicht	
Formatierung	**schlechte Namen**	
Wettläufe	**falsches Datenmodell**	Band 20, Kap. 2
Kopierte Sperren	**zu große Funktionen**	
Ungenutzte Variablen	**fehlende Zerlegung**	

Die rechte Spalte enthält alles, was Band 20 behandelt — und kein Werkzeug findet davon etwas. Eine grüne Prüfstrecke sagt, dass die bekannten Fehlerklassen fehlen; sie sagt nichts darüber, ob der Entwurf trägt.

Übungsaufgaben

(a) Richten Sie die Prüfstrecke ein.

(b) Finden Sie ein Leistungsproblem mit dem Profil.

(c) Bewerten Sie zusätzliche Prüfer.

Lösung

(a) Die Prüfstrecke.

St.	Schritt	Dauer	Wann
1	`gofmt -l` — meldet Abweichungen	**1 s**	**jede Übergabe**
2	`go vet`	6 s	jede Übergabe
3	`go build`	9 s	jede Übergabe
4	`go test`	14 s	jede Übergabe
5	**`go test -race`**	92 s	**jede Übergabe**
6	Zusatzprüfer	40 s	nächtlich

Die ersten fünf Stufen dauern zusammen gut zwei Minuten — innerhalb der Grenze aus Band 17, Kapitel 10. Der Wettlaufprüfer ist der teuerste Posten und der wichtigste; wer ihn nach hinten schiebt, findet Wettläufe erst nachts.

Einrichtung	Aufwand	
Einmalig	≈ **2 h**	**Band 21:** 16 **h**
Altlast-meldungen	≈ 30	Band 21: 400
Konfiguration	fast keine	

Zwei Stunden gegen sechzehn, dreißig Altlastmeldungen gegen vierhundert. Der Unterschied ist nicht Sprachqualität, sondern die Entscheidung, die Werkzeuge mitzuliefern und den Formatierer von Anfang an verbindlich zu machen.

(b) Ein Leistungsproblem finden.
Das Bild. Die Übertragung schafft 180 k/s statt der erwarteten 500 k/s.

	Schritt	Dauer	Befund
1	Rechenzeitprofil	3 min	8 % Last
2	**Blockierungsprofil**	3 min	**Kanal**
3	Zeile ansehen	5 min	**Puffer** 0**, ein Leser**
4	Vier Leser starten	10 min	**495 k/s**

Das Rechenzeitprofil zeigt acht Prozent Last — also kein Rechenproblem. Erst das Blockierungsprofil zeigt, wo gewartet wird, und das ist bei einem nebenläufigen Programm fast immer die richtige Frage.

Einundzwanzig Minuten von der Meldung zur Behebung — und der entscheidende Schritt war, das richtige Profil zu wählen. Wer nur Rechenzeit misst, sucht stundenlang an der falschen Stelle.

(c) Zusätzliche Prüfer bewerten.

Prüfer	Genauigkeit	Urteil
`go vet`	≈ 95 %	**immer**
Wettlaufprüfer	**100 %**	**immer**
Stilprüfer	≈ 60 %	auswählen
Komplexitätsmaße	≈ 30 %	Hinweis, keine Regel
Alle Prüfer eines Bündels	≈ 35 %	**Rauschen**

Die letzte Zeile ist der häufigste Fehler beim Einrichten: Man schaltet alle vorhandenen Prüfer ein, bekommt tausend Meldungen und schaltet nach zwei Wochen alle wieder ab. Dieselbe Rechnung wie in Band 19, Kapitel 4 und Band 21, Kapitel 10.

Die beiden ersten Zeilen genügen für den Anfang. Weitere Prüfer nimmt man einzeln auf, wenn sie im eigenen Bestand echte Funde liefern — gemessen an einer Stichprobe, nicht nach Empfehlung.

Der Wettlaufprüfer braucht Prüffälle

Voraussetzung	Erfüllt?	
Nebenläufiger Pfad geprüft	**oft nicht**	**die Lücke**
Genug Durchläufe	teils	
Realistische Last	selten	

Der Prüfer sieht nur, was ausgeführt wird — dieselbe Einschränkung wie bei den Sanitizern in Band 21. Ein Wettlauf in einem Pfad, den kein Prüffall erreicht, bleibt unentdeckt, und die grüne Meldung wiegt in falscher Sicherheit.

Die Abhilfe ist eine Prüfung, die den nebenläufigen Pfad tatsächlich belastet: hundert Goroutinen, tausend Durchläufe, mit `t.Parallel`. Zwanzig Zeilen, und der Prüfer bekommt etwas zu sehen.

Die Werkzeugdokumentation[27] beschreibt das Profil und die Prüfer; der Wettlaufprüfer[28] ist eigens dokumentiert; “Effective Go”[7] nennt die Gewohnheiten, die die Werkzeuge voraussetzen.

Kapitel 11

Prüfen

Die Tabellenprüfung

```
func TestAnlaesse(t *testing.T) {
faelle := []struct {
name    string
wert    Ergebnis
vorher *Befund
will    []Anlass
}{
{"kein Vorbefund", zahl(4.2), nil, nil},
{"genau auf Grenze", zahl(3.5), nil, nil},
{"neu ausserhalb", zahl(2.8), nil, []Anlass{Aussen}},
}
for _, f := range faelle {
t.Run(f.name, func(t *testing.T) {
got := Anlaesse(f.wert, f.vorher)
if !slices.Equal(got, f.will) {
t.Errorf("got %v, will %v", got, f.will)
}
})
}
}
```

Die Form ist in Go so verbreitet, dass sie einen Namen hat. Ihr Wert liegt in der zweiten Hälfte: Ein neuer Fall ist eine Zeile in der Tabelle, kein neuer Prüfling — und der Name erscheint bei einem Fehlschlag im Protokoll.

Was sie leistet.

Eigenschaft	Wert	
Neuer Fall	**eine Zeile**	Hemmschwelle sinkt
Fälle einzeln laufbar	**ja**	`-run`
Name im Protokoll	ja	
Nebenläufig prüfbar	**ja**	`t.Parallel`

Die erste Zeile ist die entscheidende und dieselbe Einsicht wie in Band 20, Kapitel 5: Nicht die Möglichkeit zu prüfen entscheidet, sondern die Bequemlichkeit. Wenn ein Fall eine Zeile kostet, schreibt man dreißig; wenn er zwanzig kostet, schreibt man drei.

Was Go mitliefert

Mittel	Eingebaut?	
Prüfrahmen	**ja**	**kein Zusatz nötig**
Abdeckungs-messung	ja	`-cover`
Messläufe	**ja**	`Benchmark`
Zufallsprüfung	ja	seit 1.18
Beispiele als Prüfung	**ja**	**in der Dokumentation**
Behauptungs-bibliothek	nein	absichtlich

Die vorletzte Zeile ist eine Eigenheit: Ein Beispiel in der Dokumentation wird beim Prüflauf ausgeführt und seine Ausgabe verglichen. Damit kann Dokumentation nicht veralten — eine Antwort auf das Problem aus Band 17, Kapitel 12.

Die letzte Zeile ist umstritten: Go liefert kein `assertEqual`, weil man Vergleiche mit `if` ausschreiben soll. Das ist mehr Text und liest sich beim Fehlschlag klarer — man sieht, was verglichen wurde.

Was die Abdeckung sagt

Aussage	Zutreffend?	
Nicht abgedeckt → ungeprüft	**ja**	**brauchbar**
Abgedeckt → geprüft	**nein**	nur ausgeführt
Höhere Zahl → besser	nein	Band 17, Kap. 7
100 % als Ziel	**nein**	

Die Abdeckung ist eine Aussage in eine Richtung: Was nicht ausgeführt wurde, ist sicher ungeprüft. Was ausgeführt wurde, kann trotzdem ungeprüft sein — eine Zeile ohne Zusicherung zählt genauso wie eine mit.

Deshalb ist der nützliche Gebrauch nicht die Zahl, sondern die Karte: Welche Zweige sind nie durchlaufen worden? Meist sind es die Fehlerpfade — und dort sitzen die Fehler aus Band 21, Kapitel 12.

Prüfen im selben Paket oder daneben?

	`paket`	`paket_test`
Sieht Internes	ja	nein
Prüft die Schnittstelle	teils	**ja**
Bricht bei Umbau	**ja**	**nein**
Empfehlung	für Einzelteile	**voreingestellt**

Die dritte Zeile ist der Befund aus Band 20, Kapitel 9: Prüfungen, die gegen die innere Struktur laufen, brechen bei jedem Umbau. Go bietet dafür ein eigenes Paket — ein Zeichen im Namen, und die Prüfung sieht nur, was ein Benutzer sieht.

Übungsaufgaben

(a) Prüfen Sie eine nebenläufige Stufe.

(b) Messen Sie mit einem Messlauf.

(c) Bestimmen Sie, was Attrappen ersetzt.

Lösung

(a) Eine nebenläufige Stufe prüfen.

```
func TestRechne(t *testing.T) {
ctx, ab := context.WithCancel(context.Background())
defer ab()

ein := make(chan Rohwert)
aus := rechne(ctx, ein)

go func() { ein <- roh(42); close(ein) }()

got, ok := <-aus
if !ok || got.Wert != 4.2 { t.Fatalf("got %v", got) }
if _, ok := <-aus; ok { t.Error("Kanal nicht geschlossen") }
}
```

Geprüft wird	Zeile	
Wert richtig	10	
Kanal wird geschlossen	11	**oft vergessen**
Abbruch wirkt	fehlt	**eigener Prüfling**
Kein Leck	fehlt	Kap. 6

Die zweite Zeile ist die Prüfung, die fast immer fehlt: Ein Kanal, der nicht geschlossen wird, lässt jeden Leser hängen. Eine Zeile prüft es — und deckt einen der häufigsten Fehler aus Kapitel 5 ab.

Die beiden fehlenden Zeilen gehören in eigene Prüflinge. Bei einer nebenläufigen Stufe braucht man mindestens drei: der gute Fall, der Abbruch, die Leckfreiheit.

(b) Ein Messlauf.

```
func BenchmarkAnlaesse(b *testing.B) {
w := zahl(4.2)
b.ReportAllocs()
for i := 0; i < b.N; i++ {
_ = Anlaesse(w, nil)
}
}
```

Fassung	ns/Op	Byte/Op	Belegungen
Erster Entwurf	**412**	**192**	**4**
Abschnitt vorbelegt	186	64	1
Feld statt Abschnitt	**31**	**0**	**0**

Die dritte Spalte ist die wichtigste und wird oft übersehen: Belegungen kosten nicht nur Zeit, sie erzeugen Arbeit für die Speicherbereinigung. Null Belegungen bedeutet, dass diese Funktion die Bereinigungspausen nicht verlängert.

`ReportAllocs` *ist eine Zeile und macht den Unterschied sichtbar. Wer nur Nanosekunden misst, sieht Faktor dreizehn; wer Belegungen misst, sieht auch, woher er kommt.*

(c) Was Attrappen ersetzt.

Statt	In Go	
Attrappe für Datei	`bytes.Buffer`	**eingebaut**
Attrappe für Netz	**`httptest.Server`**	**echter Bedienteil**
Attrappe für Uhr	Funktionsfeld	Band 20, Kap. 3
Attrappe für Gerät	**eigene 4-Zeilen-Umsetzung**	Kap. 3
Attrappen-rahmenwerk	selten nötig	

Die zweite Zeile ist die wertvollste: Statt den Netzverkehr nachzubilden, startet man einen echten Bedienteil auf einem freien Anschluss. Die Prüfung geht durch den vollständigen Netzstapel — und findet damit Fehler, die eine Attrappe verdeckt.

Die vierte Zeile folgt aus den kleinen Schnittstellen: Eine Schnittstelle mit einer Methode braucht eine Ersatzumsetzung von vier Zeilen. Wer große Schnittstellen baut, braucht ein Rahmenwerk — die Verbindung zwischen Kapitel 3 und diesem hier.

Prüfungen, die nur manchmal fehlschlagen

Ursache	Abhilfe	
Feste Wartezeiten	**Zustand abfragen**	nie `Sleep`
Reihenfolge angenommen	sortieren	
Echte Uhrzeit	**Zeit herein-reichen**	Band 20, Kap. 3
Freier Anschluss fest	Anschluss 0	
Gemeinsamer Zustand	**je Prüfling eigener**	

Eine Prüfung, die in einem von zwanzig Läufen fehlschlägt, wird nach zwei Wochen abgeschaltet — und mit ihr die Fehler, die sie gefunden hätte. Sprödigkeit ist deshalb kein Schönheitsfehler, sondern ein Angriff auf den gesamten Prüfbestand.

Die erste Zeile ist die häufigste Ursache in nebenläufigem Code. Statt eine Zehntelsekunde zu warten, fragt man den Zustand in kurzen Abständen ab, bis er stimmt oder eine großzügige Frist abläuft.

Bodner[1] zeigt übliche Prüfformen mit ihren Abwägungen; Kennedy[14] behandelt Messläufe und Belegungen; Fowler[8] liefert die Schritte, mit denen man Prüfbarkeit nachträglich herstellt.

Kapitel 12

Leistung und Speicherbereinigung

Was die Bereinigung kostet

Größe	Heute	
Anhaltezeit	**< 0,5 ms**	**meist** < 100 **µs**
Häufigkeit	lastabhängig	
Anteil an der Rechenzeit	≈ 5–25 %	
Vorhersagbar	**nein**	**die Grenze**

Die erste Zeile ist besser, als viele glauben — eine halbe Millisekunde ist für fast jede Anwendung unbemerkbar. Die letzte Zeile bleibt trotzdem: "meist unter hundert Mikrosekunden" ist keine Zusicherung, und eine harte Schranke verlangt eine.

Der Vergleich mit Band 21.

Schranke	C	Go	
2 ms hart	**ja**	**nein**	**Messerfassung**
50 ms weich	ja	**ja**	Übertragung
1 s	ja	ja	

Der Unterschied ist nicht "schnell" gegen "langsam", sondern "zugesichert" gegen "fast immer". Für die Messerfassung entscheidet das; für die Übertragungsschicht nicht.

Der wirksame Hebel: weniger belegen

Maßnahme	Wirkung	
Abschnitt vorbelegen	**stark**	`make(...,0,n)`
Puffer wiederverwenden	stark	`sync.Pool`
Werte statt Zeiger	**stark**	**kleine Strukturen**
Zeichenketten bauen	mittel	`strings.Builder`
Bereinigungsanteil stellen	Speicher gegen Zeit	`GOGC`

Die Reihenfolge ist nach Wirkung geordnet, und die ersten drei sind Entwurfsentscheidungen, keine Einstellungen. Wer nichts belegt, erzeugt keine Arbeit für die Bereinigung — das ist wirksamer als jede Stellschraube.

Wo die Werte liegen

	Keller	Halde
Wer entscheidet	**der Übersetzer**	**der Übersetzer**
Kosten	≈ 0	Belegung + Bereinigung
Beeinflussbar	mittelbar	
Sichtbar	`-gcflags=-m`	

Anders als in C entscheidet nicht der Programmierer, wo ein Wert liegt, sondern eine Analyse im Übersetzer: Was die Funktion nicht überlebt, bleibt auf dem Keller. Man kann es nicht anordnen — aber nachsehen und den Code so ändern, dass die Analyse zum gewünschten Ergebnis kommt.

Was einen Wert auf die Halde zwingt.

Ursache	Vermeidbar?	
Zeiger wird zurückgegeben	selten	
Wert in `interface{}`	**oft**	**häufigste Ursache**
Abschnitt wächst	ja	vorbelegen
Zu groß für den Keller	nein	
An Goroutine übergeben	teils	

Die zweite Zeile überrascht: Jeder Wert, der als leere Schnittstelle übergeben wird — etwa an eine Ausgabefunktion — landet auf der Halde. Eine Protokollzeile mit fünf Werten erzeugt fünf Belegungen, auch wenn sie nie ausgegeben wird.

Protokollierung als Leistungsproblem

Form	ns/Op	Byte/Op	Abgeschaltet
Formatiert	**840**	**264**	**310 ns**
Strukturiert	190	48	**9 ns**
Mit Stufenprüfung	190	48	**2 ns**

Die letzte Spalte ist die entscheidende: Was kostet eine Protokollzeile, die wegen der eingestellten Stufe gar nicht ausgegeben wird? Bei formatierter Ausgabe dreihundertzehn Nanosekunden — die Argumente werden ausgewertet und verpackt, bevor jemand fragt, ob sie gebraucht werden.

Bei fünfhundert Messwerten je Sekunde und drei Protokollzeilen je Wert sind das eine halbe Millisekunde je Sekunde für nichts. Strukturierte Protokollierung mit Stufenprüfung senkt das auf drei Mikrosekunden.

Übungsaufgaben

(a) Senken Sie die Belegungen.

(b) Bestimmen Sie die Wirkung von `GOGC`.

(c) Untersuchen Sie, wann Go zu langsam ist.

Lösung

(a) Belegungen senken.
Die Aufgabe. Bündeln von 500 Messwerten zu einem Paket, 1000 mal je Sekunde.

Fassung	µs/Op	Belegungen	Bereinigung
Naiv	**184**	**1503**	**22** %
Abschnitt vorbelegt	61	3	9 %
Puffer wiederverwendet	**38**	**0**	**4** %

Faktor fünf bei der Zeit — und die letzte Spalte zeigt, wo er herkommt: Der Anteil der Speicherbereinigung fällt von zweiundzwanzig auf vier Prozent. Man hat nicht die Rechnung beschleunigt, sondern die Arbeit für die Bereinigung beseitigt.

Der wiederverwendete Puffer.

```
var pakete = sync.Pool{
New:  func() any { return make([]Messwert, 0, 500) },
}

p := pakete.Get().([]Messwert)[:0]
defer pakete.Put(p)
```

Eigenschaft	Bewertung	
Wirkung	**stark**	bei hoher Frequenz
Lesbarkeit	schlechter	Typumwandlung
Fehlerquelle	**Puffer weitergegeben**	**Kap. 9**
Wann einsetzen	**nach der Messung**	

Die dritte Zeile ist die Gefahr: Wer einen zurückgegebenen Puffer noch irgendwo hält, liest später fremde Daten. Das ist genau der Fehler aus Band 21, Kapitel 3 — die Speicherbereinigung schützt nicht davor, weil der Speicher gültig ist.

(b) Die Wirkung von GOGC.

GOGC	Speicher	Bereinigung	Durchsatz
50	**120** MB	18 %	410 k/s
100	210 MB	9 %	**495** k/s
400	**680** MB	**3** %	**520** k/s
Speichergrenze	**gedeckelt**	passt sich an	**empfohlen**

Eine reine Tauschgröße: mehr Speicher gegen weniger Bereinigungsarbeit. Die letzte Zeile ist die bessere Antwort — statt eines Verhältnisses gibt man eine Speicherobergrenze an, und die Laufzeitumgebung regelt selbst.

Das ist besonders in Behältern wichtig: Ohne Obergrenze kennt die Laufzeitumgebung die Speichergrenze des Behälters nicht und wird von außen abgeschossen, statt häufiger zu bereinigen.

(c) Wann Go zu langsam ist.

Aufgabe	**Go geeignet?**	
Netzarbeit	**sehr**	**wartet ohnehin**
Textverarbeitung	ja	
Zahlenrechnen	$\approx 1{,}3\times$ C	meist genug
Enge Vektorrechnung	$\approx 3\times$ C	keine Vektorbefehle
Harte Schranken	**nein**	**Bereinigung**

Die dritte Zeile überrascht viele: Bei gewöhnlicher Rechenarbeit ist Go etwa dreißig Prozent langsamer als C — nicht Faktor drei, wie man bei einer Sprache mit Speicherbereinigung erwarten könnte. Der Übersetzer erzeugt ordentlichen Maschinencode.

Die vierte Zeile ist die echte Schwäche: Der Übersetzer nutzt Vektorbefehle kaum, und es gibt keine bequeme Möglichkeit, sie anzufordern. Wer Signalverarbeitung schreibt, ist in C oder mit einer Bibliothek besser bedient.

Die Reihenfolge, wie in Band 21

	Schritt	Aufwand	Gewinn
1	Messen	1 h	—
2	**Blockierungen prüfen**	2 h	**oft** 2–5×
3	Belegungen senken	1 d	2–5×
4	Verfahren verbessern	2 d	2–100×
5	Teil in C	Wochen	1,3–3×

Der zweite Schritt ist der Go-eigene und steht vorn, weil er bei nebenläufigen Programmen am häufigsten trägt: Ein Kanal ohne Puffer, ein einzelner Leser, eine Sperre mit Streit — Faktor drei für zwei Stunden Arbeit.

Der letzte Schritt lohnt in Go noch seltener als in Band 21: Faktor 1,3 für Wochen Arbeit, dazu die Kosten der Sprachgrenze aus Band 21, Kapitel 14. Meist ist die Antwort, den Bestandteil ganz in C zu schreiben — oder es zu lassen.

Die Werkzeugdokumentation[27] erklärt die Profilarten; Cheney[2] hat die Messpraxis am gründlichsten beschrieben; Wirth[23] erinnert daran, dass die Datenstruktur vor der Einstellung kommt.

Teil V

Das Urteil

Kapitel 13

Der zweite Entwurf in Go

Was hier zusätzlich zählt

Kriterium	Band 20	Hier
Verstehenszeit	ja	ja
Änderbarkeit	ja	ja
Prüfbarkeit	ja	ja
Größenordnung	ja	ja
Endet jede Goroutine?	—	**ja**
Wirkt der Abbruch?	—	**ja**

Zwei zusätzliche Bedingungen, beide aus Kapitel 6. Sie sind nicht ästhetisch, sondern prüfbar — und ein Entwurf, der sie nicht erfüllt, hat ein Leck, gleichgültig wie gut er sich liest.

Der Fall: der Empfänger

Der erste Entwurf.

```
func Empfange(port int) {
l, _ := net.Listen("tcp", fmt.Sprint(":", port))
for {
c, _ := l.Accept()
go func() {
d, _ := io.ReadAll(c)
speichere(entpacke(d))
c.Close()
}()
}
}
```

Schwäche	Folge	
Fehler verworfen	**stille Verluste**	**Kap. 2**
Kein Abbruch	Beenden unmöglich	
Unbegrenzte Goroutinen	**Lastspitze**	Kap. 4
`ReadAll` ohne Grenze	**Speicher**	**Angriff**
Keine Zeitschranke	hängende Verbindung	
`Close` nach Fehler	übersprungen	

Elf Zeilen, funktioniert im Versuch, und sechs Schwächen — von denen zwei ausreichen, um den Empfänger mit einer einzigen Verbindung lahmzulegen. Die vierte ist die gefährlichste: Ein Gegenüber, das nie aufhört zu senden, füllt den Speicher.

Der zweite Entwurf.

```
func Empfange(ctx context.Context, l net.Listener,
max int64, gleichzeitig int) error {
sem := make(chan struct{}, gleichzeitig)
var wg sync.WaitGroup
go func() { <-ctx.Done(); l.Close() }()

for {
c, err := l.Accept()
if err != nil {
wg.Wait()
return ctx.Err()
}
sem <- struct{}{}
wg.Add(1)
go func() {
defer func() { wg.Done(); <-sem; c.Close() }()
c.SetDeadline(time.Now().Add(30 * time.Second))
d, err := io.ReadAll(io.LimitReader(c, max))
if err != nil { melde(err); return }
if err := speichere(entpacke(d)); err != nil {
melde(err)
}
}()
}
}
```

Vierundzwanzig Zeilen statt elf. Der Abbruch wirkt über das Schließen des Lauschers — ein Kunstgriff, weil `Accept` sich nicht anders unterbrechen lässt. Die Zählgruppe wartet auf alle laufenden Verbindungen, bevor die Funktion zurückkehrt.

Die Bilanz.

	Erster	Zweiter	
Zeilen	11	**24**	+118 %
Argumente	1	4	
Schwächen	**6**	**0**	
Sauber beendbar	nein	**ja**	
Speicher unter Angriff	∞	**begrenzt**	
Verstehenszeit	34 s	**71** s	2,1×

Die letzte Zeile ist unangenehm und wird hier nicht verschwiegen: Der zweite Entwurf ist schwerer zu lesen. Das ist ein echter Preis, und er unterscheidet diesen Fall von den meisten in Band 20.

Die Antwort auf die längere Lesezeit

Weg	Wirkung	
Zurückbauen	**nein**	Schwächen kehren zurück
Zerlegen	**ja**	**Annahme, Behandlung**
Kommentar zum Abbruch	ja	drei Zeilen

Nach der Zerlegung in `Empfange` und `behandle` liegt die Verstehenszeit bei 44 Sekunden — immer noch mehr als beim ersten Entwurf, aber deutlich besser. Das ist Band 20, Kapitel 5 als Antwort auf ein Problem, das Band 20 nicht hatte.

Übungsaufgaben

(a) Prüfen Sie die sechs Bedingungen.

(b) Untersuchen Sie, was ohne Nebenläufigkeit bliebe.

(c) Bestimmen Sie, wann man aufhört.

Lösung

(a) Die sechs Bedingungen.

Bedingung	Prüfung	Ergebnis
Verstehenszeit	5 Fragen	44 s nach Umbau
Änderbarkeit	neue Grenze	ein Argument
Prüfbarkeit	`httptest`	18 **Fälle**
Größenordnung	1000 Verbindungen	495 k/s
Goroutinen enden	**Leckprüfung**	**ja**
Abbruch wirkt	eigener Prüfling	ja

Sechs Bedingungen, sechs Prüfungen — keine davon Geschmackssache. Die fünfte ist die, die Band 20 nicht kannte, und sie hat den Umbau erst nötig gemacht: Ohne Zählgruppe kehrte `Empfange` zurück, während Verbindungen noch liefen.

(b) Was ohne Nebenläufigkeit bliebe.

Schwäche	Nebenläufig?	Bliebe
Fehler verworfen	nein	**ja**
Grenze beim Lesen	nein	**ja**
Zeitschranke	nein	ja
Abbruch	ja	nein
Begrenzung	ja	nein
Zählgruppe	ja	nein

Die Hälfte der Schwächen hat mit Nebenläufigkeit nichts zu tun — ignorierte Fehler, fehlende Grenzen, fehlende Zeitschranken. Das sind die Themen aus Band 19 und Band 20, und sie fallen in jedem nebenläufigen Code doppelt auf, weil dort mehr auf einmal geschieht.

Die Lehre: Wer nebenläufigen Code überarbeitet, sollte zuerst die gewöhnlichen Schwächen beheben. Sie sind einfacher zu finden und machen die nebenläufigen Fragen danach übersichtlicher.

(c) Wann man aufhört.

Möglich wäre noch	Nötig?	
Wiederholung mit Rückzug	**ja**	**Netz fällt aus**
Zählwerte je Verbindung	ja	Betrieb
Eigener Fadenverwalter	**nein**	Kap. 4
Sperrfreie Warteschlange	nein	**kein Anlass**
Eigenes Netzprotokoll	nein	

Die ersten beiden Zeilen sind Anforderungen und gehören dazu; die letzten drei sind Reize. Der Unterschied ist, ob eine der sechs Bedingungen sie verlangt — und keine tut es.

In Go ist die Versuchung geringer als in C, weil die Sprache tiefe Eingriffe erschwert. Sie ist nicht null: Die vierte Zeile ist eine reizvolle Aufgabe, hundert Zeilen lang, und für tausend Verbindungen je Sekunde vollständig überflüssig.

Der zweite Entwurf kann schlechter lesbar sein

Band	Zeilen	Verstehen	
20, Kap. 13	+9 %	−74 %	
21, Kap. 13	+64 %	+27 %	
Hier	+118 %	**+29** %	nach Umbau

Drei Bände, drei Ergebnisse. In Python wurde der zweite Entwurf besser lesbar; in C und Go schlechter. Der Grund ist derselbe: Was hinzukommt, sind Fälle, die vorher nicht behandelt waren — Speichergrenzen, Abbruch, Beendigung.

Daraus folgt keine Kritik am zweiten Entwurf, sondern eine Warnung vor dem einfachen Maßstab: "Kürzer und klarer" ist kein Gesetz. Manchmal ist richtig einfach länger und schwerer — und trotzdem richtig.

Fowler[8] liefert die Schritte des Umbaus; die Sprichwortsammlung[20] verdichtet die Go-Haltung dazu; Ousterhout[16] begründet, warum Einfachheit nicht Kürze bedeutet.

Kapitel 14

Wann Go und wann nicht

Die vier Gründe, die tragen

Grund	Trägt?	
Viele gleichzeitige Verbindungen	**ja**	**Kap. 4**
Große Mannschaft	**ja**	kleine Sprache
Einfache Auslieferung	**ja**	eine Datei
Schnelle Einarbeitung	ja	2–5 Tage

Der zweite Grund wird selten genannt und ist in der Praxis der wichtigste: Wenn zwanzig Leute an einem Bestand arbeiten, ist eine Sprache, die man in einer Woche kennt und in der es nur einen Stil gibt, mehr wert als jede Ausdrucksstärke.

Die Gründe, die nicht tragen.

Behauptung	Trägt?	
"Go ist schnell"	teils	≈ 1,3× **C**
"Nebenläufigkeit ist gelöst"	**nein**	**Kap. 6**
"Einfach heißt fehlerfrei"	nein	
"Alle benutzen es"	nein	

Die zweite Zeile ist die verbreitetste Fehlannahme dieses Bandes: Goroutinen und Kanäle machen Nebenläufigkeit ausdrückbar, nicht sicher. Die drei Fehlerklassen aus Kapitel 6 sind alle in Go leichter zu erzeugen als in C.

Wo Go nicht passt

Aufgabe	Go?	Besser
Harte Zeitschranken	**nein**	**C, Rust, Ada**
Sehr wenig Speicher	nein	Laufzeit-umgebung
Ohne Betriebssystem	nein	
Mathematik, Vektoren	**nein**	**C, Fortran**
Oberflächen	schwach	
Datenauswertung	schwach	**Python, R**
Wissenschaftliches Rechnen	nein	

Sieben Bereiche, in denen Go nicht die richtige Wahl ist — eine längere Liste als bei den meisten Sprachen. Das ist die Kehrseite der Spezialisierung: Go wurde für einen bestimmten Zweck gebaut und ist außerhalb davon unauffällig bis ungeeignet.

Die zweite Zeile ist eine harte Grenze: Ein Go-Programm bringt eine Laufzeitumgebung von etwa zwei Megabyte mit — mehr, als der Messdatenerfasser aus Band 21 insgesamt hat.

Das Vorhaben, vollständig verteilt

Bestandteil	Schranke	Merkmal	Wahl
Messerfassung	2 **ms hart**	64 KB	**C**
Übertragung	50 ms weich	1000 **Verb.**	**Go**
Auswertung	keine	viel Fachlogik	**Python**
Oberfläche	keine	—	Web

Vier Bestandteile, vier Sprachen, und jede Wahl folgt aus einer Eigenschaft der Aufgabe. Das ist die Antwort, auf die die Bände 20 bis 22 zusammen hinauslaufen — nicht "welche Sprache", sondern "welche wofür".

Übungsaufgaben

(a) Rechnen Sie Go gegen die Alternativen.

(b) Bewerten Sie die Auslieferung.

(c) Bestimmen Sie, was an der Grenze zu beachten ist.

Lösung

(a) Go gegen die Alternativen.
Der Fall. Die Übertragungsschicht, fünf Jahre gerechnet.

	Go	C	Rust	Python
Zeilen	**3000**	8000	3400	**2100**
Erstellung (PM)	**6**	14	11	**5**
Speicherfehler	0	≈ 19	0	0
Wettlauffehler	≈ 4	≈ 7	**0**	≈ 1
Durchsatz	**495**	530	525	**78**
Über 5 J. (PM)	**19**	40	24	**17**

Python ist billiger und sechsmal langsamer — damit fällt es an der Durchsatzanforderung aus, nicht an der Kostenrechnung. Rust hat null Wettläufe und kostet fünf Personenmonate mehr; ob das lohnt, hängt davon ab, wie teuer ein Wettlauf im Betrieb ist.

Die vierte Zeile ist die ehrlichste Aussage über Go: Es beseitigt Speicherfehler und nicht die Wettläufe. Rust tut beides — und das ist ein sachliches Argument, das ein Go-Buch nennen muss.

(b) Die Auslieferung.

	Go	Python	Java
Auslieferung	**eine Datei**	Umgebung	Laufzeit
Größe	≈ 12 MB	≈ Behälter	≈ 180 MB
Fremdrechner	**nichts nötig**	Version	Laufzeit
Übersetzen für anderes System	**ein Befehl**	—	—
Startzeit	≈ **8 ms**	≈ 90 ms	≈ 400 ms

Die vierte Zeile ist im Klinikbetrieb praktisch wertvoll: Man übersetzt auf dem Entwicklungsrechner für das Zielsystem und kopiert eine Datei. Keine Abhängigkeiten, keine Versionskonflikte, keine Installationsanleitung.

Für ein Medizinprodukt kommt hinzu: Eine einzelne Binärdatei mit einer Prüfsumme ist leichter zu dokumentieren und zu validieren als eine Umgebung mit vierzig Paketen in bestimmten Versionen.

(c) An der Grenze.

Frage	Antwort	
Aufruf nach C	teuer	Band 21, Kap. 14
Faden blockiert	ja	bei blockierendem C
Zeiger nach C geben	Regeln beachten	Bereinigung
Besser	eigener Prozess	Rohr oder Steckdose

Die dritte Zeile ist die harte: Ein Go-Zeiger darf nicht dauerhaft in C gehalten werden, weil die Speicherbereinigung Objekte verschieben und freigeben kann. Es gibt Regeln dafür, und sie werden regelmäßig gebrochen.

Die letzte Zeile ist der Rat, der aus dem Vorhaben folgt: Statt die Messerfassung in denselben Prozess zu binden, läuft sie als eigenes Programm und schickt Werte über eine Verbindung. Der Grenzübertritt kostet mehr Nanosekunden und spart eine ganze Fehlerklasse.

Die ehrliche Einordnung

Frage	Antwort	
Beste Sprache?	nein	gibt es nicht
Beste für Netzdienste?	unter den besten	
Für alles brauchbar?	nein	sieben Bereiche
Lernenswert?	ja	in einer Woche

Die letzte Zeile ist ein eigenes Argument: Der Aufwand, Go zu lernen, ist so gering, dass sich die Kenntnis auch dann lohnt, wenn man es selten benutzt — schon um Code lesen zu können, den andere schreiben.

Brooks[24] erinnert daran, dass keine Sprache die wesentliche Komplexität der Aufgabe beseitigt; Dijkstra[5] legte die Grundlagen nebenläufiger Koordination; Valsorda[22] beschreibt die Sicherheitsfragen der Go-Auslieferung.

Kapitel 15

Der Stand im Jahr 2026

Was gilt und was behauptet wird

Aussage	Beleglage	Urteil
Goroutinen sind billig	eindeutig	Faktor ≈ 4000
Kleine Schnittstellen tragen	gut	Kap. 3
Gegendruck ohne Puffer	gut	
Werkzeuge werden benutzt	gut	**weil mitgeliefert**
Auslieferung ist einfach	eindeutig	
"Nebenläufigkeit gelöst"	**falsch**	**Kap. 6**
"Einfach heißt einfach"	falsch	Komplexität wandert
"Fehlerbehandlung ist gelöst"	umstritten	30 % **der Zeilen**

Die vierte Zeile ist der praktisch wertvollste Befund dieses Bandes und keine Sprachfrage: Werkzeuge, die mitgeliefert werden und ohne Einrichtung laufen, werden benutzt. Zwei Stunden Einrichtung gegen sechzehn — das ist die Lehre für jedes Vorhaben, auch in anderen Sprachen.

Fünf Entwicklungen

Erstens: Generizität ist angekommen.

Wirkung	Befund	
Bibliotheken	**schlanker**	`slices`, `maps`
Anwendungscode	wenig verändert	selten nötig
Befürchtete Überkomplexität	**blieb aus**	
Übersetzungszeit	leicht gestiegen	

Die zweite Zeile ist bemerkenswert: Vier Jahre nach der Einführung schreibt der durchschnittliche Anwendungscode kaum eigene Typparameter. Der Nutzen liegt in den Bibliotheken — genau dort, wo die zwölf Jahre Wartezeit am meisten wehgetan hatten.

Zweitens: Die Schleifenvariable ist behoben.

Bis Go 1.21	Seit 1.22	
Eine Variable je Schleife	**eine je Durchlauf**	
Häufigster Fehler	**beseitigt**	Kap. 10
Rückwärts-verträglich	über Version geregelt	

Eine seltene Entscheidung: Eine Sprachsemantik wurde geändert, weil sie mehr Fehler erzeugte als Nutzen brachte. Die Verträglichkeit wird über die in der Moduldatei genannte Sprachversion geregelt — alter Code bleibt gültig.

Drittens: Regulierung erreicht auch Go.

Anforderung	Erfüllt?	
Speichersicherheit	**ja**	empfohlene Sprache
Stückliste	eingebaut	**im Binärabbild**
Reproduzierbares Bauen	**ja**	
Abhängigkeits-prüfung	ja	`govulncheck`

Die zweite Zeile ist eine Go-Eigenheit mit regulatorischem Wert: Jede Binärdatei enthält die Liste ihrer Bestandteile samt Versionen und Prüfsummen. Für ein Medizinprodukt bedeutet das, dass die Stückliste nicht gepflegt werden muss — sie ist im Ergebnis enthalten.

Viertens: Die Nachfrage bleibt konzentriert.

Bereich	Anteil
Dienste im Netz	**hoch**
Werkzeuge, Betrieb	hoch
Anwendungen mit Oberfläche	gering
Wissenschaft	**sehr gering**

Fünfzehn Jahre nach der Veröffentlichung ist die Verteilung stabil — Go hat sich in seiner Nische durchgesetzt und ist darüber hinaus kaum gewachsen. Das ist ein Erfolg für eine Sprache, die für eine Nische entworfen wurde.

Fünftens: Werkzeuge, die Go schreiben.

Wirkung	Befund	
Erster Entwurf	schneller	**wenige Formen**
Fehler-behandlung	**gut erzeugt**	gleichförmig
Nebenläufigkeit	**fehlerhaft**	**Lecks, fehlender Abbruch**
Prüfbedarf	steigt	

Die zweite und dritte Zeile stehen im Gegensatz: Die gleichförmige Fehlerbehandlung wird gut erzeugt, weil sie wenige Formen hat. Nebenläufiger Code wird schlecht erzeugt, weil die Fehler nicht im Text sichtbar sind — eine fehlende Abbruchbehandlung sieht aus wie funktionierender Code.

Umso wichtiger sind Wettlaufprüfer und Leckprüfung aus Kapitel 10. Sie prüfen genau das, was beim Lesen nicht auffällt.

Drei Einsichten

Erstens: Weglassen ist eine Entscheidung mit Preis.

Auslassung	Gewinn	Preis
Ausnahmen	**sichtbarer Pfad**	30 % **der Zeilen**
Vererbung	**keine Hierarchien**	Wiederholung
Überladung	Lesbarkeit	Namenswucher
Kleine Sprache	2–5 **Tage**	wenig Ausdruck

Vier Zeilen, acht Spalteneinträge, und keiner davon ist umsonst. Wer nur die mittlere Spalte liest, hält Go für überlegen; wer nur die rechte liest, für armselig. Beide Spalten sind wahr.

Zweitens: Nebenläufigkeit wird ausdrückbar, nicht sicher.

Fehlerklasse	In Go	
Wettlauf	möglich	Prüfer findet
Verklemmung	möglich	teils erkannt
Goroutinenleck	**leichter als in C**	**kein Werkzeug**
Speicherfehler	**ausgeschlossen**	der echte Gewinn

Die letzte Zeile ist der Fortschritt gegenüber Band 21 — die häufigste Fehlerklasse überhaupt verschwindet. Die dritte Zeile ist der Rückschritt, den man dafür in Kauf nimmt, und er wird selten genannt.

Drittens: Das Handwerk bleibt dasselbe.

Aus Band 20	Gilt hier?	
Aufgabe verstehen	**unverändert**	
Daten zuerst	unverändert	
Fehlersuche als Verfahren	unverändert	**plus Profile**
Fremden Code lesen	**leichter**	**kleine Sprache**
Wann aufhören	plus zwei Kriterien	

Dreimal dasselbe Ergebnis über drei Bände: Fünf von sechs Fertigkeiten sind sprachunabhängig. Nur die vierte Zeile fällt hier besser aus — eine kleine Sprache ist leichter zu lesen, und das war ihr erklärter Zweck.

Was dieser Band nicht behauptet

Behauptung	Aufgestellt?	
Go ist die beste Sprache	**nein**	**Kap. 14**
Weglassen ist immer richtig	**nein**	vier Preise
Nebenläufigkeit ist gelöst	nein	
Diese Zahlen sind Konstanten	nein	**Größen-ordnungen**

Die zweite Zeile ist die wichtigste. Weglassen ist eine Entscheidung, keine Tugend — und jede der acht Auslassungen hat einen Anwendungsbereich, außerhalb dessen sie schadet. Kapitel 1 nennt für jede das Gegenbeispiel.

Eine Sprache mit wenigen Mitteln
zwingt dazu, das Problem zu verstehen.
Eine mit vielen erlaubt es,
das Verständnis zu vertagen.

— die Bilanz dieses Bandes.
Beides hat seinen Preis.
Nur der erste wird sofort bezahlt.

Die Beiträge zu Typparametern[26] erklären die Rücknahme einer Auslassung; Cox[4] dokumentiert die Entwurfsentscheidungen; die Spezifikation[11] bleibt die verbindliche Quelle.

Teil VI

Anhang

Kapitel A

Regeln und Kenngrößen

Die Regeln

Fehler	Zusammenhang hinzufügen, ursprünglichen behalten
Schnittstellen	Beim Aufrufer, so klein wie möglich
Goroutinen	**Wer sie startet, weiß, wann sie endet**
Kanäle	Der Sender schließt, nie der Empfänger
Puffer	**Vorgabe null — Gegendruck erhalten**
Nullwert	Typen so entwerfen, dass er brauchbar ist
Abschnitte	Teilstücke nach außen mit begrenzter Kapazität
Pakete	Ein Geheimnis je Paket, keine Zyklen
Abhängigkeiten	**Erst die Standardbibliothek prüfen**
Prüfstrecke	`vet` und Wettlaufprüfer bei jeder Übergabe

Die wichtigsten Zahlen

Größe	Wert	Kapitel
Keller Goroutine gegen Faden	Faktor ≈ 4000	4
Goroutine erzeugen	≈ 0,3 µs	4
Praktische Obergrenze	≈ 10^6	4
Kanaloperation	≈ 100 ns	5
Sperre gegen Kanal	Faktor ≈ 14	5
Schnittstellenaufruf	≈ 2,4 ns	3
Anteil Fehlerbehandlung	≈ 30 %	2
Zeit bis Ursache	≈ 6 min	2
Anhaltezeit Bereinigung	< 0,5 ms	12
Anteil Bereinigung	5–25 %	12
Go gegen C, Rechenarbeit	≈ 1,3×	12
Einarbeitung	2–5 Tage	1
Prüfstrecke einrichten	≈ 2 h	10
Wettlaufprüfer, Laufzeit	×5–10	6

Alle Werte sind Größenordnungen. Die Verhältnisse sind belastbarer als die absoluten Zahlen — und auf Verhältnissen beruhen die Argumente dieses Bandes.

Die acht Auslassungen, bilanziert

Fehlt	Gewinn	Preis
Ausnahmen	**sichtbarer Pfad**	30 % Zeilen
Vererbung	**keine Hierarchien**	Wiederholung
Klassen	wenige Begriffe	
Konstruktoren	**Nullwert nutzbar**	nicht immer möglich
Makros	**lesbarer Text**	keine Erweiterung
Operator-überladung	Lesbarkeit	**Mathematik leidet**
Implizite Umwandlung	**keine Überraschung**	mehr Text
Generizität bis 2022	zurück-genommen	zwölf Jahre

Acht Auslassungen, acht Gewinne, sieben Preise — die Zusammenfassung dieses Bandes auf einer Seite. Keine Zeile ist umsonst zu haben.

Kapitel B

Landkarte und Glossar

Wo was steht

Kap.	Thema	Verweise
1	Was fehlt und warum	**Band 20, Kap. 8**
2	Fehler als Werte	**Band 21, Kap. 5**
3	Schnittstellen	**Band 17, Kap. 4**
4	Goroutinen	**Band 4**
5	Kanäle	Band 4, Band 21, Kap. 9
6	Wo es schiefgeht	**Band 21, Kap. 10**
7	Pakete	**Band 21, Kap. 11**
8	Zusammensetzung	Band 17, Band 20, Kap. 14
9	Der Nullwert	**Band 21, Kap. 3**
10	Werkzeuge	**Band 21, Kap. 10**
11	Prüfen	**Band 20, Kap. 9**
12	Leistung	Band 21, Kap. 2
13	Der zweite Entwurf	**Band 20, Kap. 13**
14	Wann Go	**Band 21, Kap. 14**
15	Stand 2026	Band 19

Begriffe

Abschnitt	Fenster auf ein Feld: Zeiger, Länge, Kapazität
Einbettung	Feld ohne Namen; Methoden werden hochgereicht
Gegendruck	Bremsen des Senders durch blockiertes Senden
Gestaltprüfung	Erfüllung einer Schnittstelle ohne Erklärung
Goroutine	Von der Laufzeitumgebung verwalteter Ablauf
Goroutinenleck	Goroutine, die nie endet
Kanal	Typisierte Übergabestelle zwischen Goroutinen
Nullwert	Anfangswert eines Typs ohne Zuweisung
Tabellenprüfung	Prüffälle als Datenzeilen statt als Funktionen
Zählgruppe	Zähler, der auf das Ende mehrerer Goroutinen wartet

Kapitel C

Kommentierte Quellen

[1] Bodner, J.: *Learning Go*. 2. Aufl., O'Reilly 2024.

Die beste Einführung für Umsteiger. Behandelt ausdrücklich die Umgewöhnung — warum man keine Hierarchien baut und was stattdessen tritt. Aktuell bis einschließlich der Typparameter.

[2] Cheney, D.: Beiträge und Vorträge zu Go. Fortlaufend.

Frei verfügbar und über Jahre gepflegt. Die Beiträge zur Fehlerbehandlung und zur Messpraxis sind die gründlichsten Texte zu diesen Fragen, die es gibt — gründlicher als jedes Buch.

[3] Cox, R.: *Our Software Dependency Problem*. 2019.

Der Aufsatz zur Frage, was eine fremde Abhängigkeit kostet. *Nüchtern, mit Belegen, ohne Alarmismus — und mit einer brauchbaren Prüfliste vor der Aufnahme.*

[4] Cox, R.: Beiträge zum Sprachentwurf und zu Modulen. Fortlaufend.

Erklärt die Entscheidungen von innen: warum Module so aussehen, wie sie aussehen, und warum die Typparameter zwölf Jahre gebraucht haben. Selten, dass ein Sprachentwurf so offen begründet wird.

[5] Dijkstra, E.: *Cooperating Sequential Processes*. 1965.

Die Grundlagenschrift zur nebenläufigen Koordination — Semaphore, gegenseitiger Ausschluss, Verklemmung. Alles, was Kapitel 6 behandelt, steht hier zum ersten Mal.

[6] Donovan, A., Kernighan, B.: *The Go Programming Language*. Addison-Wesley 2015.

Das Standardwerk, in der Tradition von [K&R]. *Vollständig, knapp, mit durchdachten Beispielen. Älter als die Typparameter und in allem anderen unverändert gültig.*

[7] *Effective Go*. Go-Dokumentation, seit 2009.

Die Gewohnheiten der Sprache auf etwa vierzig Seiten — Benennung, Zerlegung, Nebenläufigkeitsmuster. Frei verfügbar und in zwei Stunden gelesen.

[8] Fowler, M.: *Refactoring*. 2. Aufl., Addison-Wesley 2018.

Der Katalog der Umgestaltungsschritte, sprachunabhängig. Für Kapitel 13 die Grundlage — und die Erinnerung, dass ohne Prüfnetz kein Umbau stattfindet.

[9] Gerrand, A.: *Go Slices: usage and internals*. 2011.

Der Text, der Abschnitte erklärt *— die drei Zahlen, das geteilte Feld, die Folgen für* `append`*. Wer ihn gelesen hat, macht die Fehler aus Kapitel 9 nicht mehr.*

[10] *The Go Memory Model*. Sprachdokumentation.

Legt fest, welche Zusicherungen zur Sichtbarkeit zwischen Goroutinen gelten. Kurz, formal und die einzige verbindliche Antwort auf die Frage, was ein Wettlauf ist.

[11] *The Go Programming Language Specification*. Fortlaufend.

Etwa neunzig Seiten für die vollständige Sprache — lesbar an einem Nachmittag. Dass das möglich ist, ist selbst ein Argument in der Sache dieses Bandes.

[12] Griesemer, R., Pike, R., Thompson, K.: Go FAQ und Entwurfsdokumente. Fortlaufend.

Begründet jede Auslassung einzeln. *Wer wissen will, warum es keine Vererbung und keine Ausnahmen gibt, findet hier die Antwort der Urheber statt der Vermutung Dritter.*

[13] Hoare, C. A. R.: *Communicating Sequential Processes*. CACM 21(8), 1978.

Die theoretische Grundlage der Kanäle. Anspruchsvoll und lohnend — man sieht, dass Go hier eine vierzig Jahre alte Idee aufgegriffen und praktisch gemacht hat.

[14] Kennedy, W., u. a.: *Ultimate Go Notebook*. Ardan Labs 2021.

Gründlich in den Entwurfsfolgen: Wert oder Zeiger, Belegungen, Speicheranordnung. Näher an der Maschine als die übrigen Go-Bücher — die Brücke zu Band 21.

[15] Lamport, L.: *Time, Clocks, and the Ordering of Events*. CACM 21(7), 1978.

Begründete den Begriff der Ordnung von Ereignissen in nebenläufigen Systemen. Ohne diesen Begriff lässt sich nicht sagen, was ein Wettlauf überhaupt ist.

[16] Ousterhout, J.: *A Philosophy of Software Design*. Yaknyam Press 2018.

Fordert tiefe Schnittstellen — wenig nach außen, viel dahinter. Dieselbe These wie in Band 20 und 21, und in Go am unmittelbarsten umsetzbar.

[17] Parnas, D.: *On the Criteria To Be Used in Decomposing Systems into Modules*. CACM 15(12), 1972.

Fünf Seiten, die den Modulbegriff begründet haben. *Für Kapitel 7 unmittelbar anwendbar: Ein Paket verbirgt ein Geheimnis, sonst ist es keines.*

[18] Pike, R.: *Concurrency Is Not Parallelism*. Vortrag, 2012.

Beendet eine Verwechslung in dreißig Minuten. *Nebenläufigkeit ist eine Entwurfsform, Parallelität eine Ausführungsform — danach redet man präziser über beides.*

[19] Pike, R.: *Go at Google: Language Design in the Service of Software Engineering*. 2012.

Erklärt, aus welchen Erfahrungen mit großen Beständen die Entscheidungen entstanden. Das wichtigste Dokument, um Go zu verstehen statt es nur zu benutzen.

[20] Pike, R.: *Go Proverbs*. Vortrag, 2015.

Zwanzig Sätze, die die Haltung verdichten — "ein wenig Kopieren ist besser als ein wenig Abhängigkeit" und ähnliche. Merksätze, keine Regeln, und als solche brauchbar.

[21] *Go Concurrency Patterns*. Vorträge und Beiträge, seit 2012.

Die wiederkehrenden Bauformen: Verarbeitungsketten, Aufteilen und Zusammenführen, Abbruch, Zeitschranken. Kapitel 5 folgt ihnen.

[22] Valsorda, F.: Beiträge zu Sicherheit und Kryptographie in Go. Fortlaufend.

Von jemandem, der die Kryptographie der Standardbibliothek gepflegt hat. Behandelt auch die Fragen der Auslieferung und der Abhängigkeitsprüfung.

[23] Wirth, N.: *Algorithms + Data Structures = Programs*. Prentice Hall 1976.

Der Titel ist die These und enthält eine Rangordnung. Sie gilt auch hier: Die Datenstruktur kommt vor der Einstellung des Bereinigungsanteils.

[24] Brooks, F.: *No Silver Bullet*. IEEE Computer 20(4), 1987.

Trennt wesentliche von zufälliger Komplexität. *Keine Sprache beseitigt die erste — Go beseitigt einen Teil der zweiten, und mehr zu behaupten wäre unredlich.*

[25] *Error Handling and Go* und *Working with Errors in Go 1.13*. Go-Blog, 2015 und 2019.

Erklären Kette, `Is` und `As`. Kurz und für Kapitel 2 die maßgebliche Quelle — die Fehlerbehandlung hat sich seit 1.13 erheblich verbessert.

[26] *An Introduction to Generics*. Go-Blog, 2022.

Erklärt die Typparameter und ihre Grenzen. Bemerkenswert ist die Zurückhaltung: Der Text rät ausdrücklich davon ab, sie zu verwenden, wo konkrete Typen genügen.

[27] Werkzeugdokumentation: `go vet`, `pprof`, `go test`, `govulncheck`.

Frei verfügbar, kürzer als erwartet, und ohne Einrichtungsaufwand nutzbar. Zwei Stunden genügen für eine vollständige Prüfstrecke — Kapitel 10 rechnet es vor.

[28] *Data Race Detector*. Go-Dokumentation.

Beschreibt den Wettlaufprüfer und seine Grenzen. Wichtigster Satz: Er meldet keine Fehlalarme — was er meldet, ist ein Wettlauf. Und er sieht nur, was ausgeführt wird.

Wenn Sie nur drei Bücher kaufen

[6]	**Donovan, Kernighan**	**Für die Sprache**
[1]	**Bodner**	Für die Umgewöhnung
[14]	**Kennedy**	**Für die Entwurfsfolgen**

Donovan und Kernighan lehren die Sprache, Bodner die Umstellung, Kennedy das, was darunter geschieht. Auffällig ist, wie viel hier nicht Buchform hat: Die besten Texte zu Fehlerbehandlung, Abschnitten und Nebenläufigkeit sind Beiträge und Vorträge.

> *Zwei Ergänzungen, beide frei verfügbar und beide kurz: [19] Pike, um zu verstehen, warum Go so aussieht. Und [18] — dreißig Minuten, nach denen man Nebenläufigkeit und Parallelität nicht mehr verwechselt.*